ವಿಶ್ವೇಶ್ವರಯ್ಯ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ, ಬೆಳಗಾವಿ

VISVESVARAYA TECHNOLOGICAL UNIVERSITY - BELAGAVI

BANGALORE INSTITUTE OF TECHNOLOGY

K.R.ROAD, V.V. PURA, BENGALURU -560 004

Department of Information Science and Engineering

BIS657E

Software Testing Automation Laboratory Manual

VI- Semester

2024-25

Prepared By:
Dr. Hema Jagadish
Associate Professor
Dept. of ISE, BIT

BANGALORE INSTITUTE OF TECHNOLOGY

VISION:

Establish and develop the Institute as the Centre of higher learning, ever abreast with expanding horizon of knowledge in the field of Engineering and Technology with entrepreneurial thinking, leadership excellence for life-long success and solve societal problems.

MISSION:

- Provide high quality education in the Engineering disciplines from the undergraduate through doctoral levels with creative academic and professional programs.
- Develop the Institute as a leader in Science, Engineering, Technology, Management and Research and apply knowledge for the benefit of society.
- Establish mutual beneficial partnerships with Industry, Alumni, Local, State and Central Governments by Public Service Assistance and Collaborative Research.
- Inculcate personality development through sports, cultural and extracurricular activities and engage in social, economic and professional challenges.

Bangalore Institute of Technology

K R Road, VV Puram, Bangalore- 560004

Department of Information Science and Engineering

VISION:

Empower every student to be innovative, creative and productive in the field of Information Technology by imparting quality technical education, **developing Professional Skills** and inculcating human values.

MISSION:

- To evolve continually as a centre of excellence in offering quality Information Technology **Education**.
- To nurture the students to meet the global competency in industry for **Employment**.
- To promote collaboration with industry and academia for constructive interaction to empower **Entrepreneurship**.
- To provide reliable, contemporary and integrated technology to support and facilitate **Life Long Learning.**

PROGRAM EDUCATIONAL OBJECTIVES

- Uplift the students through Information Technology **Education.**
- Provide exposure to emerging technologies and train them to **Employable** in Multi-disciplinary industries.
- Motivate them to become good professional Engineers and **Entrepreneur.**
- Inspire them to prepare for **Higher Learning and Research.**

PROGRAMME SPECIFIC OUTCOMES (PSOs)

- To provide our graduates **with Core Competence in Information Technology and Management.**
- To prepare our graduates with **relevant skills for higher Education, Entrepreneurship, Professional career & Social values.**

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Bangalore Institute of Technology

K. R. Road, V.V. Pura, Bengaluru 560004

Department of Information Science and Engineering

Software Testing Automation LABORATORY (BIS657E)

PRE-REQUISITES:

Basic Understanding of Software Engineering concepts.
Basic Programming Skills.
Fundamental Database Knowledge.
Understanding of Operating Systems & Computer Networks.

COURSE OBJECTIVES :

- To introduce the features, components, and benefits of the Katalon and Selenium Studio
- To understand the Test case design, Test execution and Test data management
- To understand the concepts of Test automation
- To understand the Test scenario development

COURSE OUTCOMES (CO)

On the completion of this laboratory course, the students will be able to:

CO1: Understand Katalon studio & Selenium architecture & key features of Katalon studio & Selenium automation tool

CO2: Apply Selenium & Katalon to build Test suites and run tests in different browsers

CO3: Analyze the test case design, Test case execution & test data management

CO4: Develop test scenarios, test cases and modules using the automation tool

		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
BIS657E	CO1	2											2
	CO2	3											2
	CO3	3	3										2
	CO4	2	2	3									3

		PSO1	PSO2
BIS657E	CO1	2	2
	CO2	3	3
	CO3	3	3
	CO4	3	3

SOFTWARE TESTING AUTOMATION LABORATORY

Subject Code: BIS657 **CIE Marks: 50**
Hours/Week: 0:0:2:0 **Exam Hours: 03**
Total Hours: 24

List of Programs

Sl. No.	Name of Experiment
1.	**Installation of Katalon Studio:** Installation and Setup, Web Locators & Techniques, Selectors Hub and browsers dev tools
2.	**Automation Modes in Katalon Studio :** Katalon Studio record mode, Katalon Studio manual mode. Katalon Studio Keywords and Katalon Studio script mode
3.	**Exploring Katalon Recorder Tool:** Recorder for Chrome browser, Firefox, Explorer. Play testcases in recorder
4.	**Exploring Web Spy & Web Object Repository:** web spy & object repository. spy on a web object in a browser, Object locators & strategies
5.	**Exploring the Predefined pre-Defined Web UI Keyboards:** browser keywords. Windows keywords, checkbox keywords, combo box, textbox. file upload. screenshot, scroll down and scroll up button related, alerts frames, mouse hover. drag and drop and keystroke keywords.
6.	**Installation of Selenium:** Setting up Selenium RC and Selenium Web Driver.
7.	**Designing Testcases:** Test Case design and Conversion of Mapping and Templates using selenium tool and Manual test cases mapping with Selenium test cases.
8.	**Using Selenium IDE:** Develop a test suite containing minimum 4 test cases.
9.	**Test Suite:** Conduct a test suite for any two web sites
10.	**Test Scripts:** Develop and test a program to login a specific web page using Selenium test scripts
11.	**Test Scripts:** Develop and test a program to provide total number of objects present available on the page using selenium test scripts
12.	**Practical Exercise and Wrap-Up:** Build Test suit with suitable application and complete end to end automation process, Discussion on Best Practices and Q&A

SOFTWARE TESTING AUTOMATION LABORATORY

Subject Code: BIS657E **CIE Marks: 50**
Hours/Week: 0:0:2:0 **Total Hours: 24**

Evaluation Criteria

Lab Write-up and Execution rubrics for Daily Conduction (Max: 30 marks)

	Task	Good	Average
a.	Write-up **(10 Marks)**	Detailed knowledge of katalon & selenium automation concepts to solve the given problem. **(10 Marks)**	Moderate knowledge of katalon & selenium automation concepts to solve the given problem. **(6 Marks)**
b.	Execution & Testing **(10 Marks)**	Execution of Testcases & modules in the automation tool and testing the testcases. **(10 Marks)**	Partial execution of test cases & modules in the automation tool and testing the Testcases. **(6 Marks)**
c.	Result Analysis & comparison **(5 Marks)**	Analysis of Testcases for different results & comparing expected & actual output. **(5 Marks)**	Partial analysis of Testcases for different results & comparing expected & actual output. **(3 Marks)**
d.	Record **(5 Marks)**	Documentation of the concepts with all possible results. **(5 Marks)**	Documentation of the concepts with partial results. **(3 Marks)**

Lab Write-up and Execution rubrics for Internals (Max: 50 marks)

	Task	Good	Average
a.	Write up **(10 Marks)**	Detailed knowledge of katalon & selenium automation concepts to solve the given problem. **(10 Marks)**	Moderate knowledge of katalon & selenium automation concepts to solve the given problem. **(6 Marks)**
b.	Execution & Testing **(10 Marks)**	Execution of Testcases & modules in the automation tool and testing the testcases. **(10 Marks)**	Partial execution of test cases & modules in the automation tool and testing the Testcases. **(6 Marks)**
c.	Result Analysis & comparison **(10 Marks)**	Analysis of Testcases for different results & comparing expected & actual output. **(5 Marks)**	Partial analysis of Testcases for different results & comparing expected & actual output. **(3 Marks)**
d.	Viva Voce **(20 Marks)**	Complete understanding of automation tool katalon & selenium testcases. **(20 Marks)**	Partial understanding of both practical and theoretical concepts. **(10 Marks)**

SOFTWARE TESTING AUTOMATION LABORATORY

Subject Code: BIS657E **CIE Marks: 50**
Hours/Week: 0:0:2:0 **Total Hours: 24**

Lesson Planning/Schedule of Experiments

Sl. No	Name of Experiment	WEEK
1	**Installation of Selenium:** Setting up Selenium RC and Selenium Web Driver.	Week1
2	**Designing Testcases:** Test Case design and Conversion of Mapping and Templates using selenium tool and Manual test cases mapping with Selenium test cases.	Week2
3	**Using Selenium IDE:** Develop a test suite containing minimum 4 test cases. **Test Suite:** Conduct a test suite for any two web sites	Week3
4	**Test Scripts:** Develop and test a program to login a specific web page using Selenium test scripts	Week4
5	**Test Scripts:** Develop and test a program to provide total number of objects present available on the page using selenium test scripts	Week5
6	**LAB TEST – 1**	Week6
7	**Practical Exercise and Wrap-Up:** Build Test suit with suitable application and complete end to end automation process, Discussion on Best Practices and Q&A	Week7
8	**Installation of Katalon Studio:** Installation and Setup, Web Locators & Techniques, Selectors Hub and browsers dev tools	Week8
9	**Automation Modes in Katalon Studio :** Katalon Studio record mode, Katalon Studio manual mode. Katalon Studio Keywords and Katalon Studio script mode	Week9
10	**Exploring Katalon Recorder Tool:** Recorder for Chrome browser, Firefox, Explorer. Play testcases in recorder	Week10
11	**Exploring Web Spy & Web Object Repository:** web spy & object repository. spy on a web object in a browser, Object locators & strategies	Week11
12	**Exploring the Predefined pre-Defined Web UI Keyboards:** browser keywords. Windows keywords, checkbox keywords, combo box, textbox. file upload. screenshot, scroll down and scroll up button related, alerts frames, mouse hover. drag and drop and keystroke keywords.	Week12
13	**LAB TEST – 2**	Week13

SOFTWARE TESTING AUTOMATION LABORATORY

Subject Code: BIS657E **Hours/Week: 0:0:2**
CIE Marks: 50 **Total Hours: 24**

INDEX

Sl. No	Contents	Page No.	Date of execution	Date of Submission	Marks(30)				Total Marks	Staff Signature
					Write-up CO1 & CO4 (10 Marks)	Execution & Testing CO2 (10 Marks)	Result Analysis & Comparison CO3 (5 Marks)	Record CO1 (5 Marks)		
1	Installation of Selenium: Setting up Selenium RC and Selenium Web Driver.									
2	Designing Testcases: Test Case design and Conversion of Mapping and Templates using selenium tool and Manual test cases mapping with Selenium test cases.									
3	Using Selenium IDE: Develop a test suite containing minimum 4 test cases.									
4	Test Suite: Conduct a test suite for any two web sites									
5	Test Scripts: Develop and test a program to login a specific web page using Selenium test scripts									
6	Test Scripts: Develop and test a program to provide total number of objects present available on the page using selenium test scripts									
7	Practical Exercise and Wrap-Up: Build Test suit with suitable application and complete end to end automation process, Discussion on Best Practices and Q&A									
8	Installation of Katalon Studio: Installation and Setup, Web Locators & Techniques, Selectors Hub and browsers dev tools									
9	Automation Modes in Katalon Studio : Katalon Studio record mode, Katalon Studio manual mode. Katalon Studio Keywords and Katalon Studio script mode									
10	Exploring Katalon Recorder Tool: Recorder for Chrome browser, Firefox, Explorer. Play testcases in recorder									
11	Exploring Web Spy & Web Object Repository: web spy & object repository. spy on a web object in a browser, Object locators & strategies									
12	Exploring the Predefined pre-Defined Web UI Keyboards: browser keywords. Windows keywords, checkbox keywords, combo box, textbox.file upload. screenshot, scroll down and scroll up button related, alerts frames, mouse hover. drag and drop and keystroke keywords.									
				Total:						
				Average:						
								Final Daily Conduction Marks : _____/30 Marks	**Signature**	

	LAB INTERNAL MARKS					
TEST-I (50 Marks)	Write up (10 Marks) CO1	Execution & Testing (10 Marks) CO2	Result Analysis & comparison (10 Marks) CO3	Viva (20 Marks) CO2	T1 Marks	Signature
TEST-II (50 Marks)	Write up (10 Marks) CO1	Execution (10 Marks) CO2	Result Analysis & comparison (10 Marks) CO3	Viva (20 Marks) CO2	T2 Marks	Signature
	Total = (T1+T2) Marks Obtained = ------ *0.2					
	Final Internal Marks:					

Final IA Marks		
Daily Conduction	Max- 30	
Lab Internal Marks	Max- 20	
TOTAL	Max- 50	
Signature of the faculty		

Program 1

Installation of Katalon Studio: Installation and Setup, Web Locators & Techniques, Selectors Hub and browsers dev tools.

Katalon Studio is an automation testing tool that provides an integrated environment for testing Web, API, Mobile, and Desktop applications. It is built on top of Selenium and Appium, making automation easier without requiring deep programming skills.

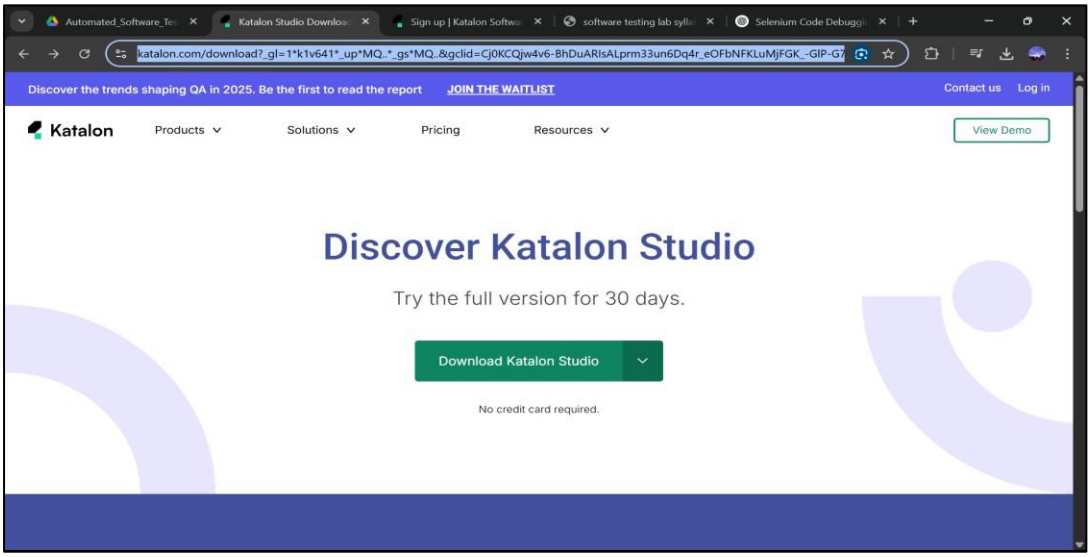
Use Cases of Katalon Studio

- **Web Testing** – Automates web applications across different browsers (Chrome, Firefox, Edge, Safari).
- **Mobile App Testing** – Supports Android & iOS automation using Appium.
- **API Testing** – Automates REST & SOAP API testing with built-in request validation.
- **Desktop App Testing** – Supports Windows desktop application automation.
- **Data-Driven Testing** – Runs tests with multiple data sets (Excel, CSV, Databases).
- **CI/CD Integration** – Works with Jenkins, GitHub Actions, Azure DevOps for automated pipelines.
- **Cross-Browser Testing** – Ensures compatibility across multiple browsers.
- **Test Reporting & Analytics** – Generates logs, screenshots, and integrates with TestOps, JIRA, Slack.
- **AI-Powered Self-Healing** – Automatically fixes broken element locators.
- **Regression & Functional Testing** – Automates repetitive testing for application stability.

1. Installation & Setup of Katalon Studio

Step 1: Download Katalon Studio

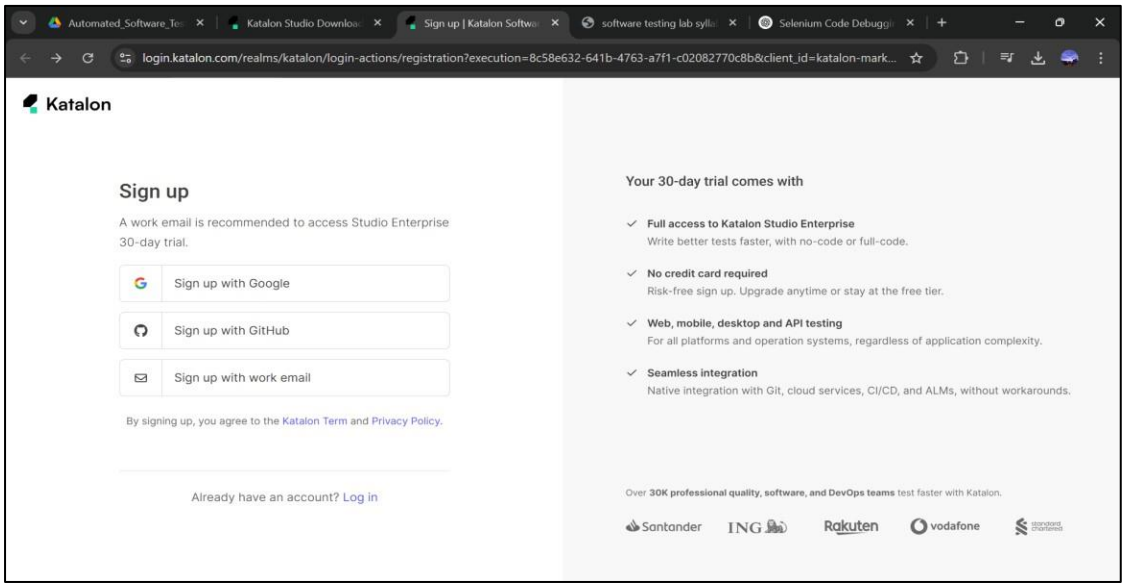
- Visit the official Katalon website and download the latest version for your OS.
- Link : https://katalon.com/download?_gl=1*k1v641*_up*MQ..*_gs*MQ..&gclid=Cj0KCQjw4v6-BhDuARIsALprm33un6Dq4r_eOFbNFKLuMjFGK_-GlP-G7EjGLM3j8kXUI_isxqXysZQaAsmREALw_wcB



- Install Katalon Studio by following the on-screen instructions.

Step 2: Create a Free Katalon Account

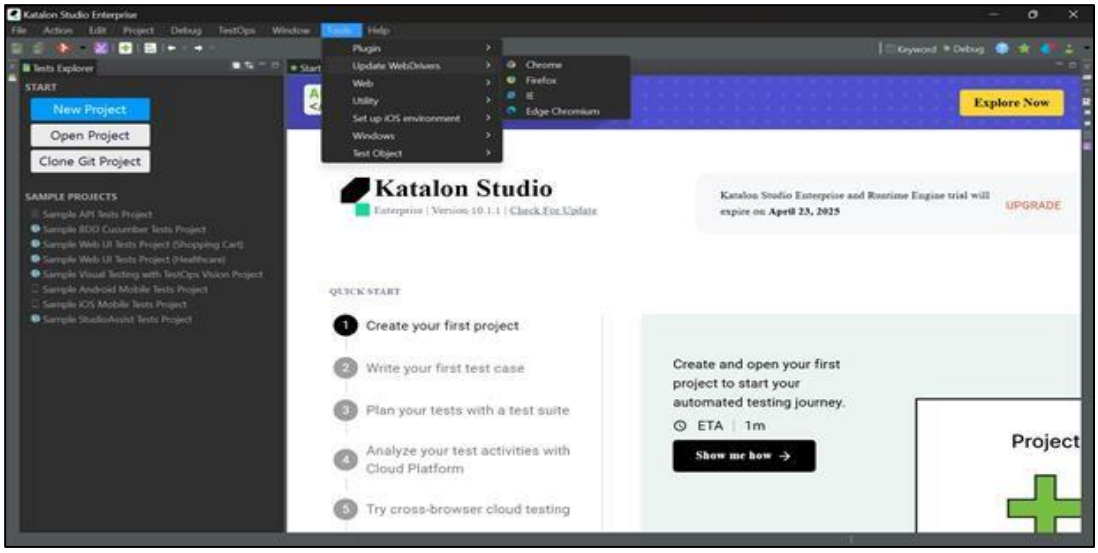
- Sign up at Katalon TestOps.
- Link : https://login.katalon.com/realms/katalon/login-actions/registration?client_id=katalon-marketing&tab_id=qKuLcFbTyFA&client_data=eyJydSI6Imh0dHBzOi8va2F0YWxvbi5jb20vc2V0dXAiLCJydCI6ImNvZGUifQ



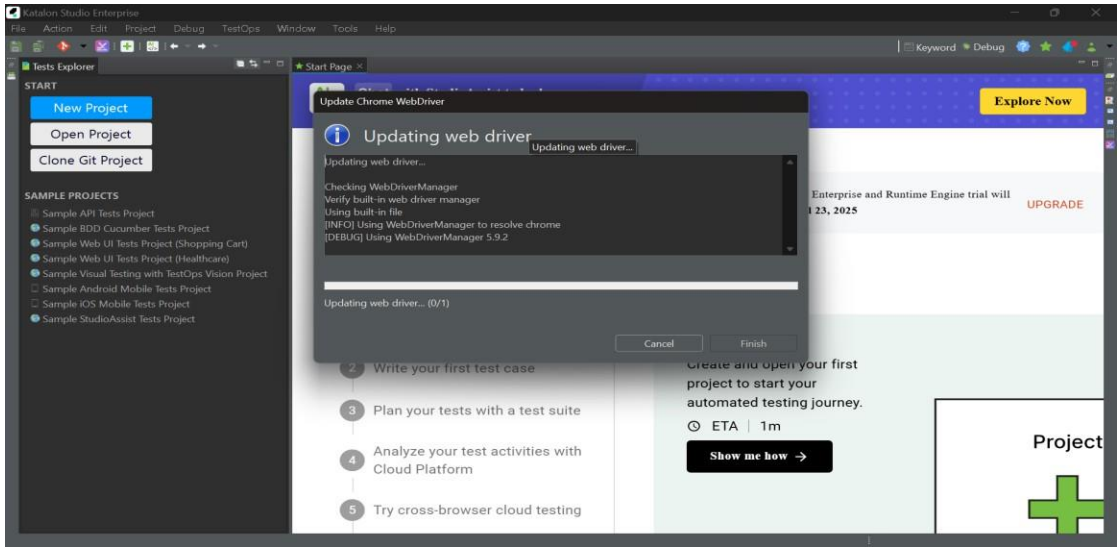
- Activate Katalon Studio with your credentials. – choose login with browser after you run katalon studio for the first time.

Step 3: Set Up Web Drivers

- Katalon automatically detects the latest **Chrome, Firefox, Edge, and Safari drivers**.
- To manually update WebDrivers:
- Go to **Katalon Studio → Tools → Web**



- Select **Update WebDrivers** for Chrome/Firefox/Edge.



2. Web Locators & Techniques

Locators help in identifying web elements for automation. Katalon provides multiple locator strategies:

Common Locator Strategies in Katalon Studio

Locator Type	Syntax in Katalon
By ID	WebUI.setText(findTestObject('Page_Login/txt_username'), 'testUser')
By Name	WebUI.click(findTestObject('Page_Login/btn_Login'))
By XPath	WebUI.verifyElementPresent(findTestObject('Object Repository/Page_Home/lbl_Dashboard'), 10)
By CSS Selector	WebUI.click(findTestObject('Object Repository/Page_Login/btn_Login'))

Example: Automating a Login Test:

```
import com.kms.katalon.core.webui.keyword.WebUiBuiltInKeywords as WebUI

// Open browser

WebUI.openBrowser('https://www.saucedemo.com/')

// Enter username

WebUI.setText(findTestObject('Object Repository/Page_Login/txt_username'),
'standard_user')

// Enter password

WebUI.setText(findTestObject('Object Repository/Page_Login/txt_password'),
'secret_sauce')

// Click login button

WebUI.click(findTestObject('Object Repository/Page_Login/btn_Login'))

// Verify login success

WebUI.verifyElementPresent(findTestObject('Object
Repository/Page_Home/lbl_Dashboard'), 10)

// Close browser

WebUI.closeBrowser()
```

HOW TO RUN IT?

Open Katalon Studio & Create a New Project

- **Launch Katalon Studio**.
- Click on **File** → **New** → **Project**.
- Enter a **Project Name** and select **Web** as the project type.
- Click **OK** to create the project.

Create a New Test Case

- In the **Test Explorer** panel, **right-click** on **Test Cases** → Select **New Test Case**.
- Give the test case a name (e.g., `LoginTest`).
- Click **OK**.

Write Your Test Script

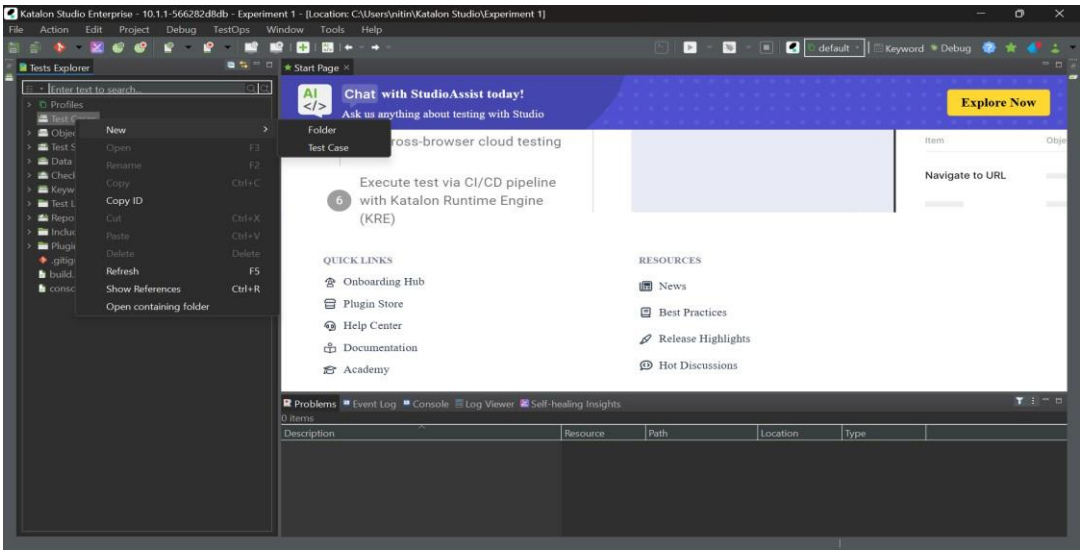
- Open the created test case and switch to the **Script Mode**.
- Copy & paste the Selenium-based script into **Katalon's Groovy-based format**.

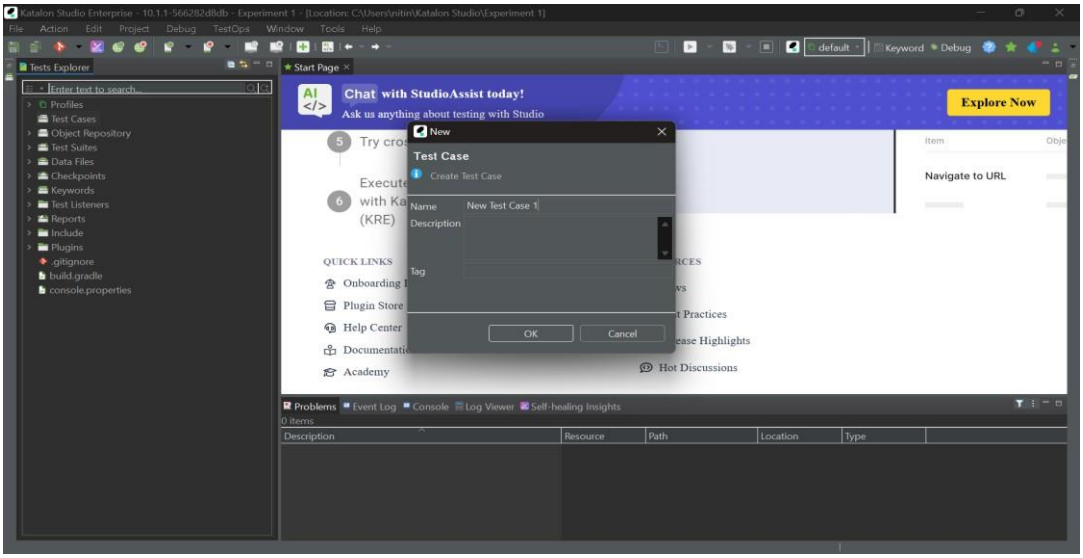
Add Test Objects in Object Repository

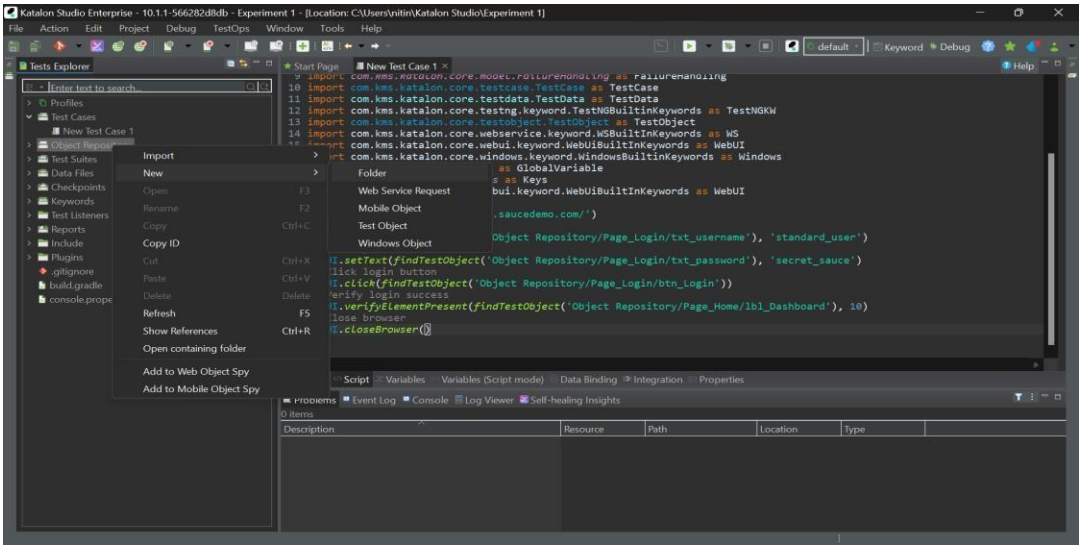
- **Go to Object Repository** → Click **New Folder** (e.g., `Page_Login`).
- Click **New** → **Test Object** and name it (`txt_username`, `txt_password`, `btn_Login`).
- Use the **Object Spy** or Autoselect or manually add **XPath/CSS selectors**.
- Click **Save**.

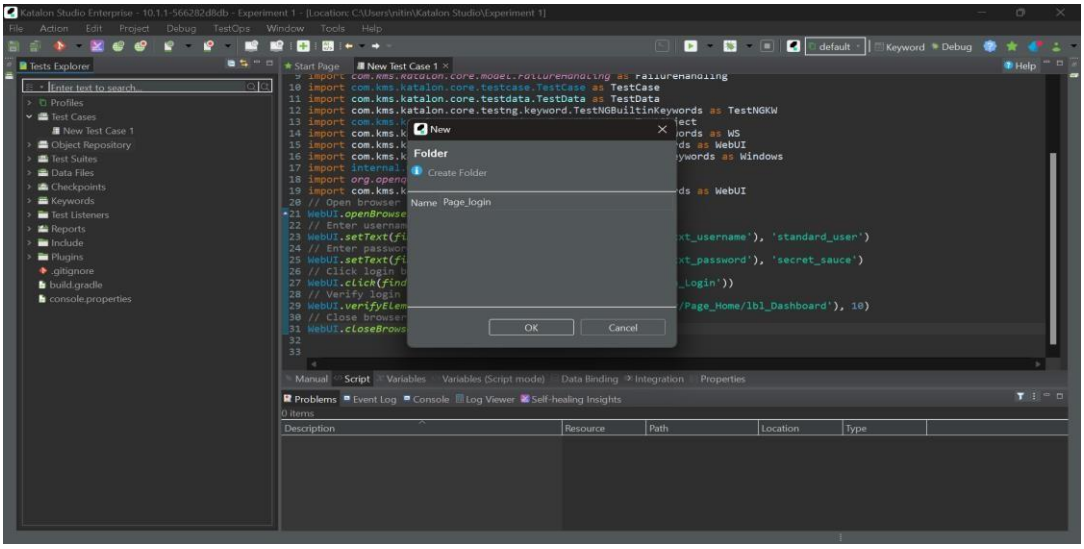
Execute the Test Case

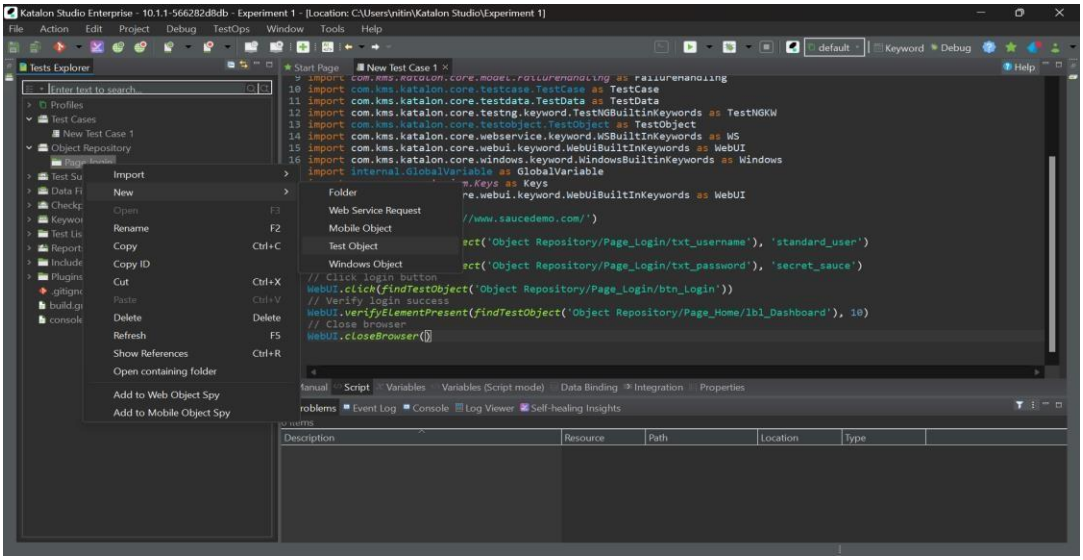
- Click **Run** (Green Play Button ▶).
- Select **Chrome/Firefox** as the browser.
- Katalon Studio will **open the browser, execute the script, and display results** in the **Log Viewer**.

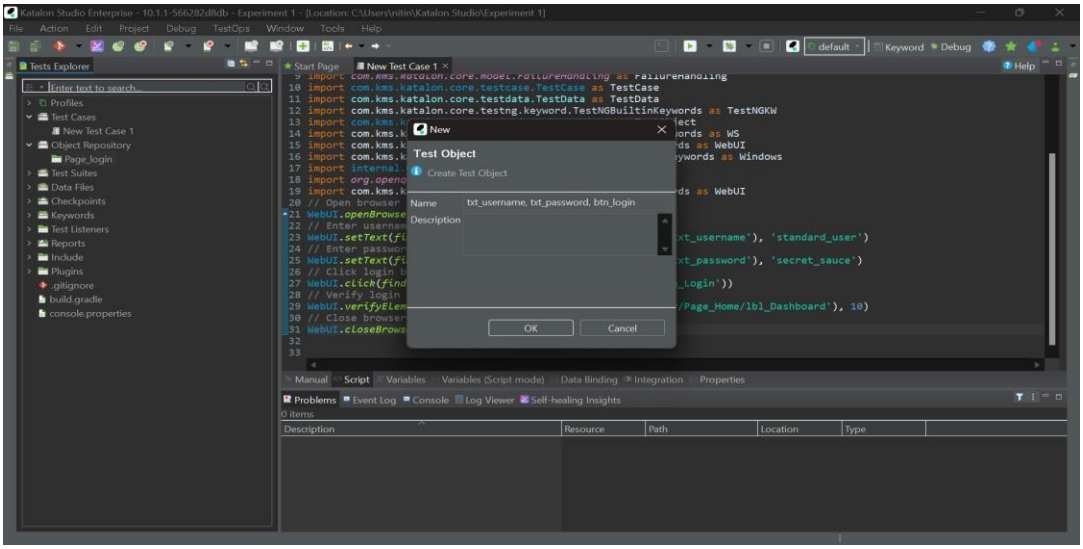

Katalon Studio Enterprise - 10.1.1-566282d8db - Experiment 1 - [Location: C:\Users\nitin\Katalon Studio\Experiment 1]
File Action Edit Project Debug TestOps Window Tools Help
Tests Explorer
Start Page
Chat with StudioAssist today!
Ask us anything about testing with Studio
Explore Now
New
Folder
Test Case
Copy ID
Refresh
Show References
Open containing folder
Execute test via CI/CD pipeline with Katalon Runtime Engine (KRE)
Navigate to URL
QUICK LINKS
Onboarding Hub
Plugin Store
Help Center
Documentation
Academy
RESOURCES
News
Best Practices
Release Highlights
Hot Discussions
Problems
Event Log
Console
Log Viewer
Self-healing Insights

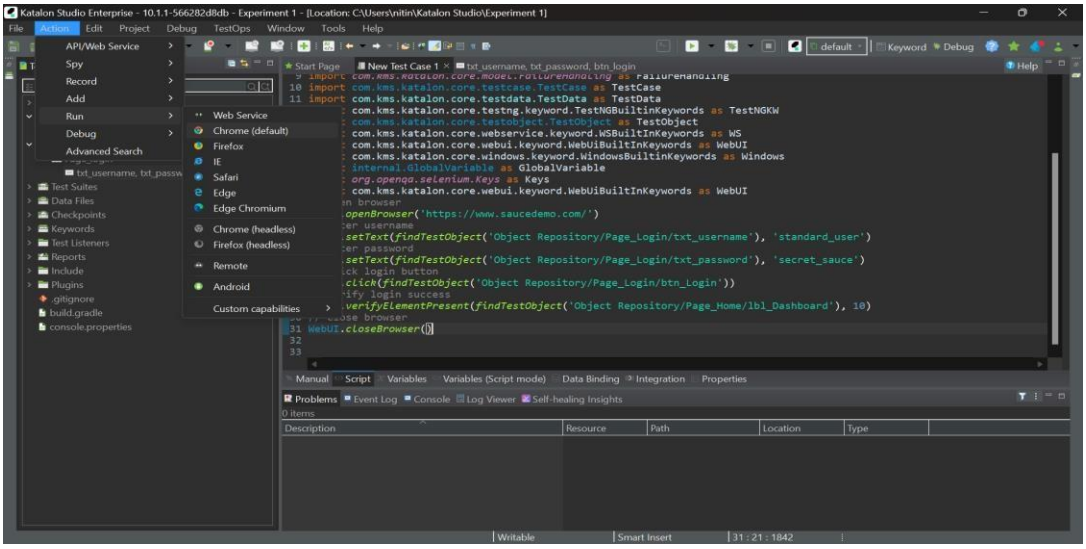

Katalon Studio Enterprise - 10.1.1-566282d8db - Experiment 1 - [Location: C:\Users\nitin\Katalon Studio\Experiment 1]
Tests Explorer
Start Page
Chat with StudioAssist today!
Ask us anything about testing with Studio
Explore Now
New
Test Case
Create Test Case
Name
New Test Case 1
Description
Tag
OK
Cancel
Navigate to URL
QUICK LINKS
Onboarding Hub
Plugin Store
Help Center
Academy
Hot Discussions
Profiles
Test Cases
Object Repository
Test Suites
Data Files
Checkpoints
Keywords
Test Listeners
Reports
Include
Plugins
.gitignore
build.gradle
console.properties

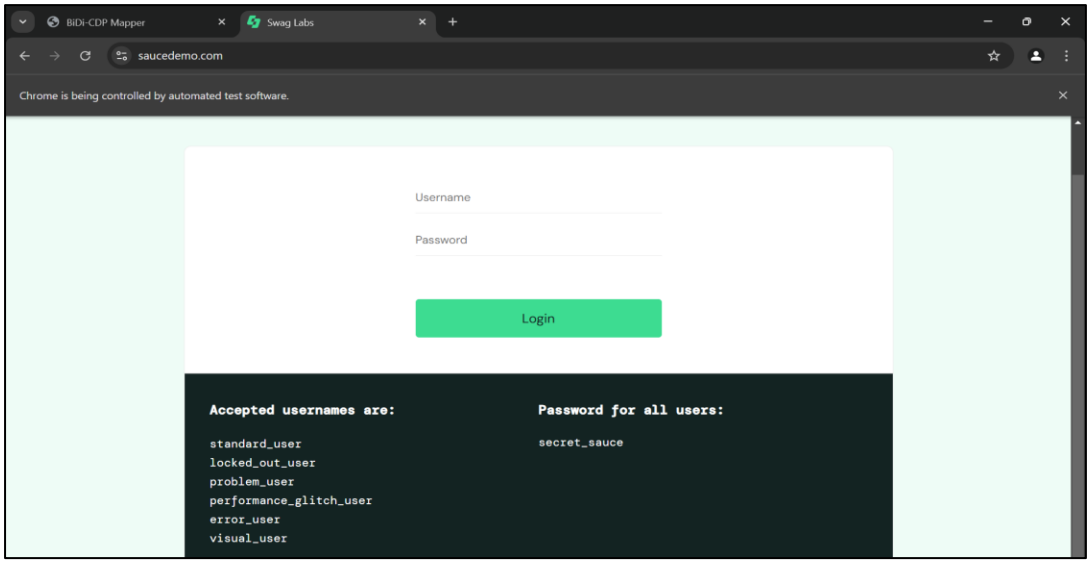

Katalon Studio Enterprise - 10.1.1-566282d8db - Experiment 1 - [Location: C:\Users\nitin\Katalon Studio\Experiment 1]
Tests Explorer
Start Page
New Test Case 1
Import
New
Folder
Web Service Request
Mobile Object
Test Object
Windows Object
Copy ID
Refresh
Show References
Open containing folder
Add to Web Object Spy
Add to Mobile Object Spy
import com.kms.katalon.core.testcase.TestCase as TestCase
import com.kms.katalon.core.testdata.TestData as TestData
import com.kms.katalon.core.testng.keyword.TestNGBuiltinKeywords as TestNGKW
import com.kms.katalon.core.testobject.TestObject as TestObject
import com.kms.katalon.core.webservice.keyword.WSBuiltInKeywords as WS
import com.kms.katalon.core.webui.keyword.WebUiBuiltInKeywords as WebUI
import com.kms.katalon.core.windows.keyword.WindowsBuiltinKeywords as Windows
Script
Variables
Variables (Script mode)
Data Binding
Integration
Properties
Problems
Event Log
Console
Log Viewer
Self-healing Insights


Katalon Studio Enterprise - 10.1.1-566282d8db - Experiment 1 - [Location: C:\Users\nitin\Katalon Studio\Experiment 1]
File Action Edit Project Debug TestOps Window Tools Help
Tests Explorer
Start Page
New Test Case 1
New
Folder
Create Folder
Name Page_login
OK
Cancel
Manual Script Variables Variables (Script mode) Data Binding Integration Properties
Problems Event Log Console Log Viewer Self-healing Insights
Description Resource Path Location Type


Katalon Studio Enterprise - 10.1.1-566282d8db - Experiment 1 - [Location: C:\Users\nitin\Katalon Studio\Experiment 1]
Import
New
Rename F2
Copy Ctrl+C
Copy ID
Cut Ctrl+X
Paste Ctrl+V
Delete Delete
Refresh F5
Show References Ctrl+R
Open containing folder
Add to Web Object Spy
Add to Mobile Object Spy
Folder
Web Service Request
Mobile Object
Test Object
Windows Object


Katalon Studio Enterprise - 10.1.1-566282d8db - Experiment 1 - [Location: C:\Users\nitin\Katalon Studio\Experiment 1]
New
Test Object
Create Test Object
Name txt_username, txt_password, btn_login
Description
OK
Cancel





3. SelectorsHub for XPath & CSS Selector

SelectorsHub is a browser extension that helps generate XPath and CSS selectors easily.

Installation:

1. Install from SelectorsHub.
2. Add to Chrome or Firefox.
3. Enable in DevTools (F12 → SelectorsHub Tab).

Usage in Katalon Studio

- Inspect the web element.
- Copy the optimized XPath or CSS selector from SelectorsHub.
- Use it in Katalon's Object Repository for element identification.

Example XPath from SelectorsHub:

//input[@id='user-name']

Usage in Katalon:

WebUI.setText(findTestObject('Object Repository/Page_Login/txt_username'), 'standard_user')

4. Using Browser DevTools for Locators

Steps to Use DevTools (Chrome)

1. Open DevTools:
   - Right-click on the element → Click Inspect OR
   - Press F12 or Ctrl + Shift + I (Windows) / Cmd + Option + I (Mac).
2. Find an element:
   - Click the Elements tab.
   - Use Ctrl + Shift + C (Windows) or Cmd + Shift + C (Mac) to inspect an element.
3. Copy XPath or CSS Selector:
   - Right-click the element → Copy → Copy XPath or Copy Selector.
4. Test XPath in Console:
   - Open the Console tab (F12 → Console).
   - Run: [JAVASCRIPT:
     - $x("//input[@id='user-name']")
     - If correct, it will highlight the element.

Program 2

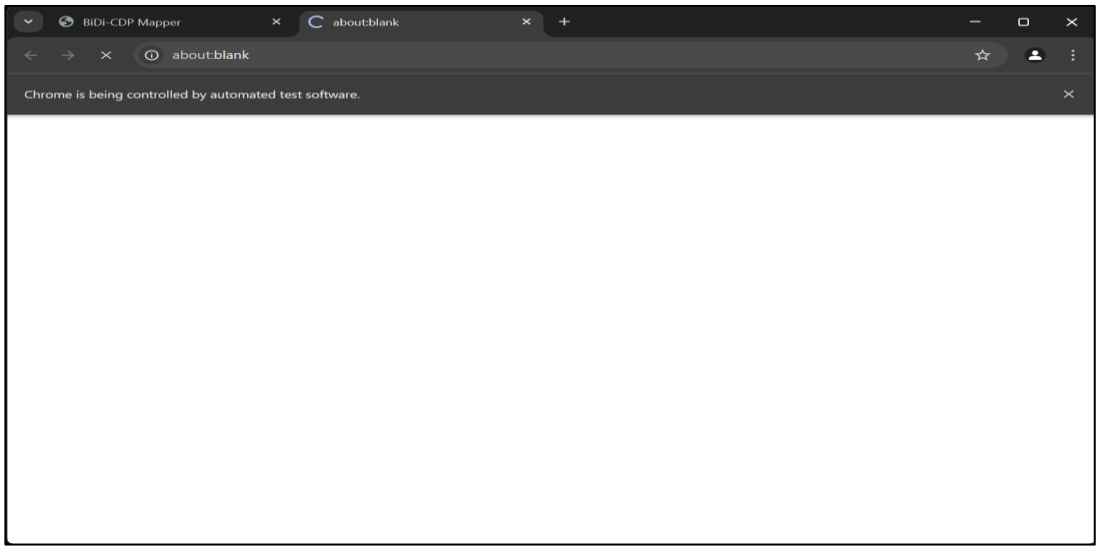
Automation Modes in Katalon Studio : Katalon Studio record mode, Katalon Studio manual mode. Katalon Studio Keywords and Katalon Studio script mode.

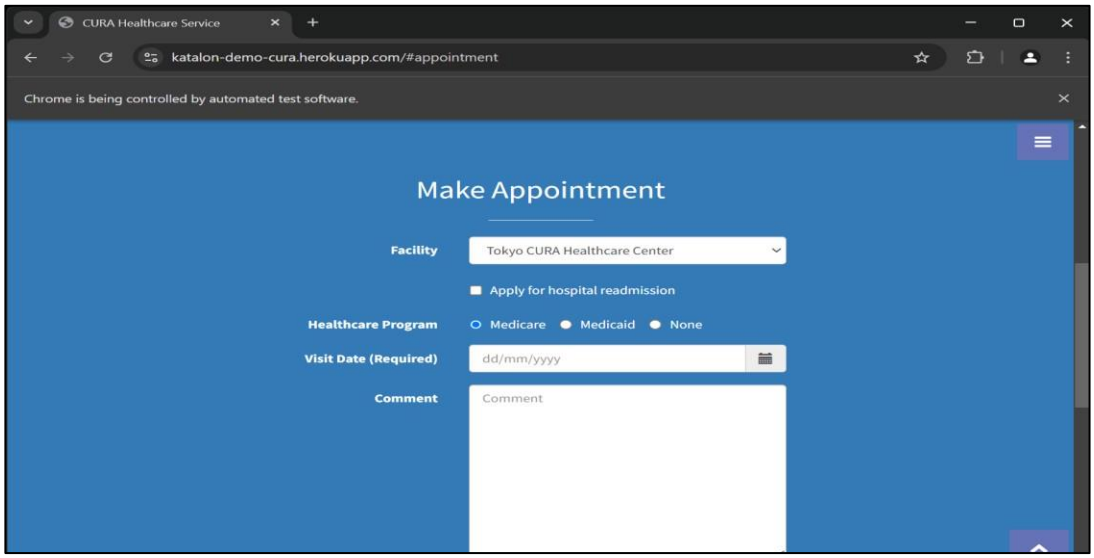
Katalon Studio offers multiple automation modes to help users create, manage, and execute automated test scripts efficiently. Below are the different automation modes and instructions on how to use them:

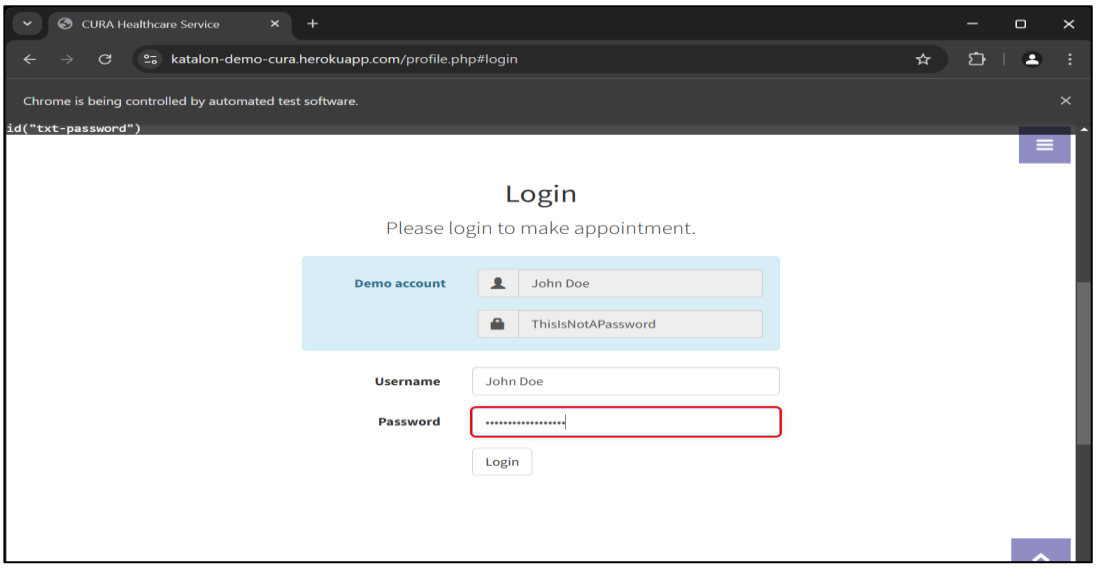
Instructions:

1. **Open Katalon Studio** and create a new test case.
2. Click on **"Record Web"** (for web applications) or **"Record Mobile"** (for mobile applications).
3. Enter the URL of the web application or select the mobile application under test.
4. Click **"Start"**, then perform actions on the application (clicking buttons, entering text, etc.).
5. Katalon Studio will record actions and display them in the test case editor.
6. Click **"Stop"** once done, then review the recorded steps.
7. Click **"Save"** to generate the test script.

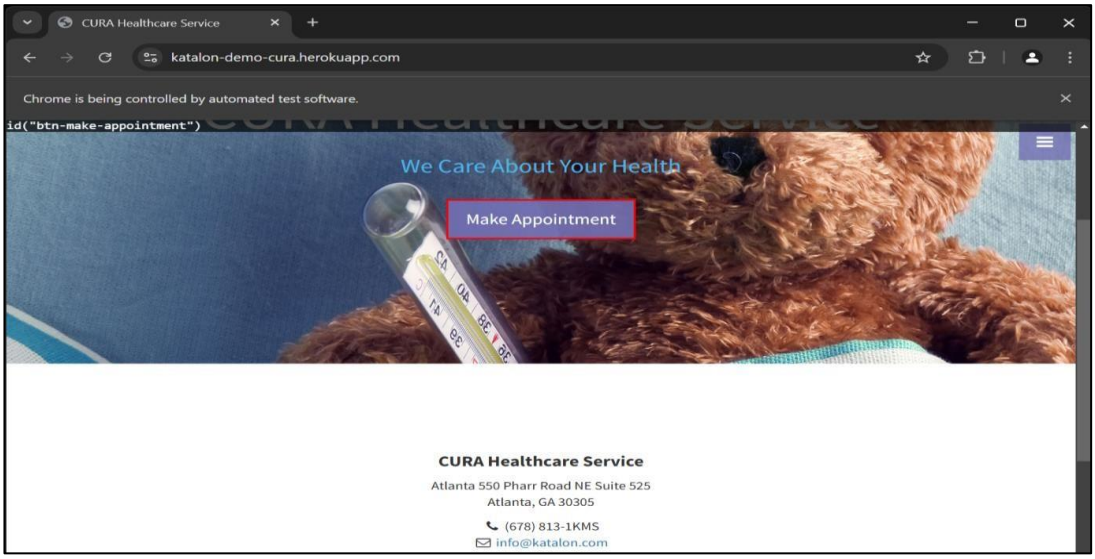
- Best for beginners and quick test case creation.

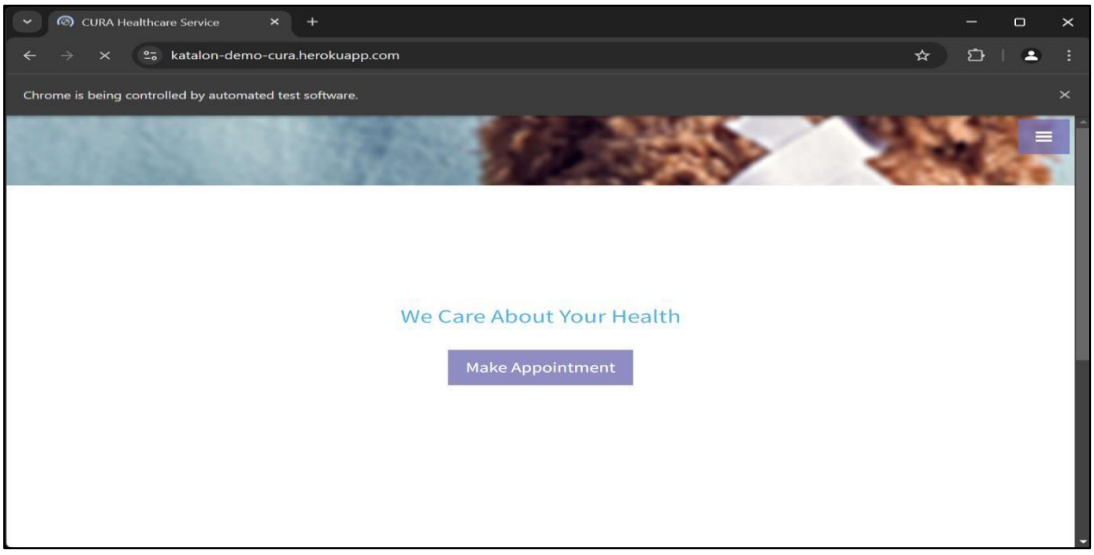


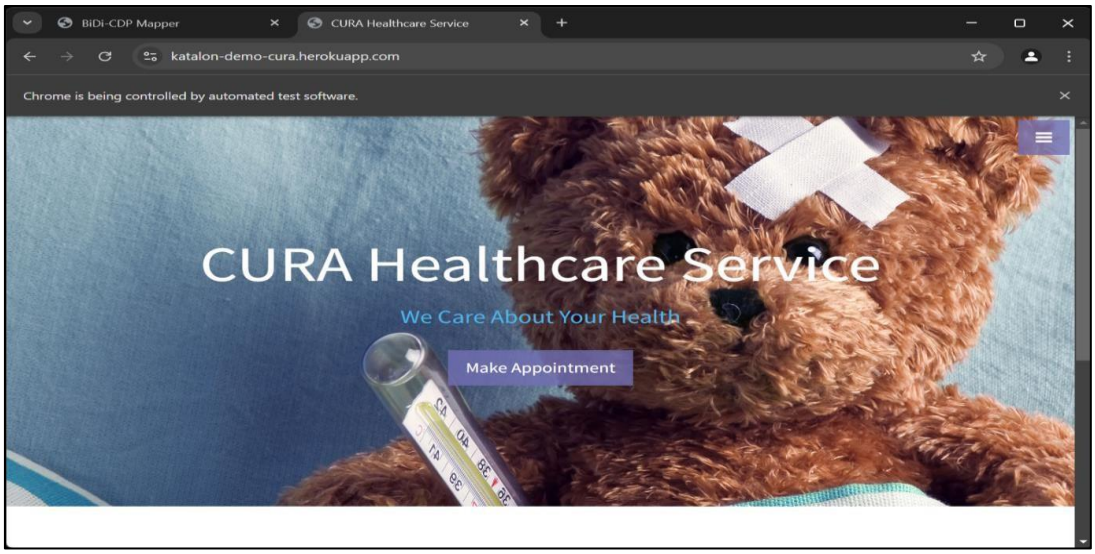

CURA Healthcare Service
katalon-demo-cura.herokuapp.com/#appointment
Chrome is being controlled by automated test software.
Make Appointment
Facility
Tokyo CURA Healthcare Center
Apply for hospital readmission
Healthcare Program
Medicare
Medicaid
None
Visit Date (Required)
dd/mm/yyyy
Comment
Comment


CURA Healthcare Service
katalon-demo-cura.herokuapp.com/profile.php#login
Chrome is being controlled by automated test software.
id("txt-password")
Login
Please login to make appointment.
Demo account
John Doe
ThisIsNotAPassword
Username
John Doe
Password
Login

‘


CURA Healthcare Service
katalon-demo-cura.herokuapp.com
Chrome is being controlled by automated test software.
id("btn-make-appointment")
We Care About Your Health
Make Appointment
CURA Healthcare Service
Atlanta 550 Pharr Road NE Suite 525
Atlanta, GA 30305
(678) 813-1KMS
info@katalon.com


CURA Healthcare Service
katalon-demo-cura.herokuapp.com
Chrome is being controlled by automated test software.
We Care About Your Health
Make Appointment



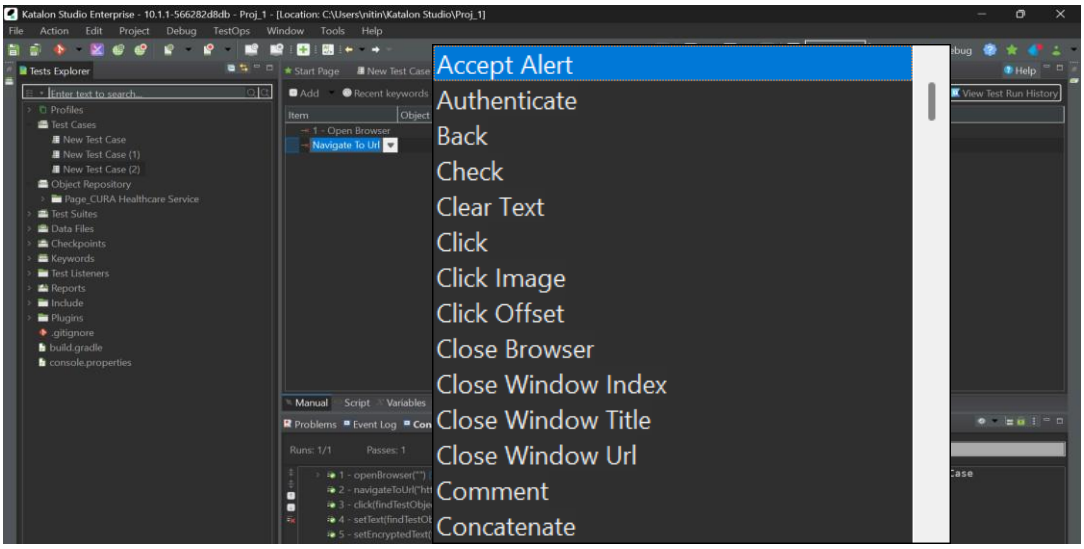
2. Katalon Studio Manual Mode

Purpose: Allows users to manually build test cases using a graphical interface without scripting.

Instructions:

1. **Create a new test case** in Katalon Studio.
2. Switch to **"Manual Mode"** in the test case editor.
3. Click **"Add"** to insert test steps manually.
4. Select the necessary **keywords (actions)** from the keyword library, such as:
   - Click
   - Set Text
   - Verify Element Present
5. Enter the required parameters for each step, like object references or input values.
6. Arrange test steps as needed using drag-and-drop.
7. Click **"Run"** to execute the test case.

- Ideal for testers with little to no coding experience.



3. Katalon Studio Keywords

Purpose: Uses built-in or custom keywords to simplify and reuse test steps across multiple test cases.

Instructions:

1. Open Katalon Studio and navigate to **"Keywords"** in the **Test Explorer**.
2. Click **"New > Keyword"** and create a new Groovy class file.
3. Define a custom keyword using Groovy scripting. Example:

```
package customKeywords
import com.kms.katalon.core.annotation.Keyword
class MyKeywords {
    @Keyword
    def printMessage(String message) {
        println message  }
}
```

4. Save the file and use the custom keyword in a test case:

```
import customKeywords.MyKeywords as MyKeywords
MyKeywords.printMessage("Hello, Katalon!")
```

5. Execute the test case to verify functionality.

4. Katalon Studio Script Mode

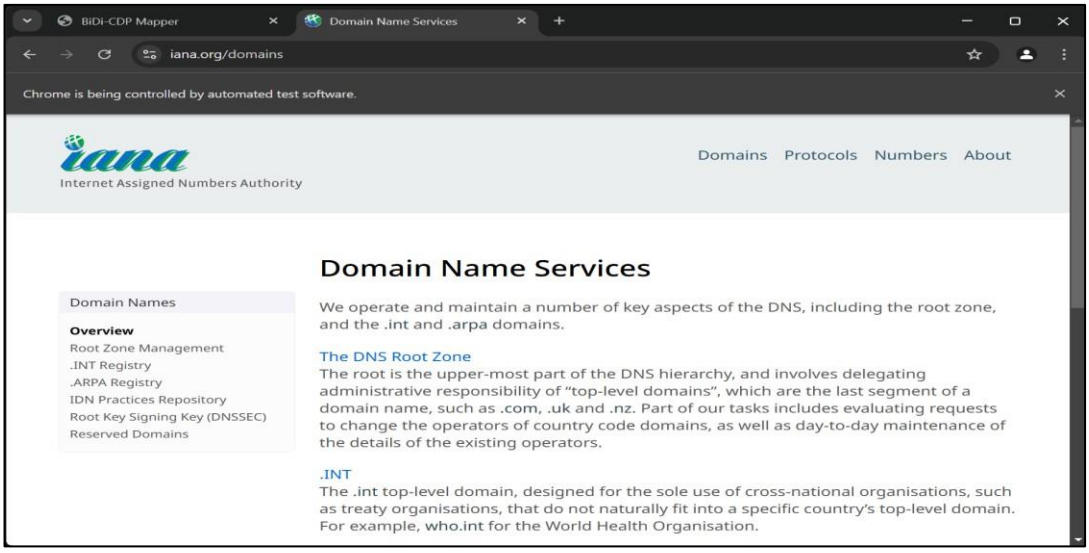
Purpose: Allows users to write and modify test scripts using Groovy/Java-based scripting.

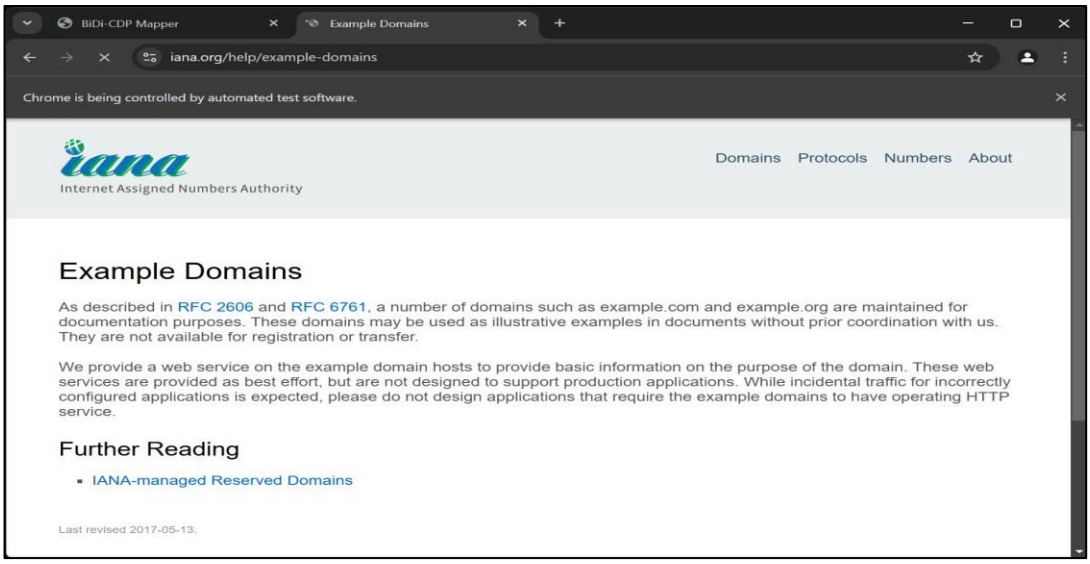
Instructions:

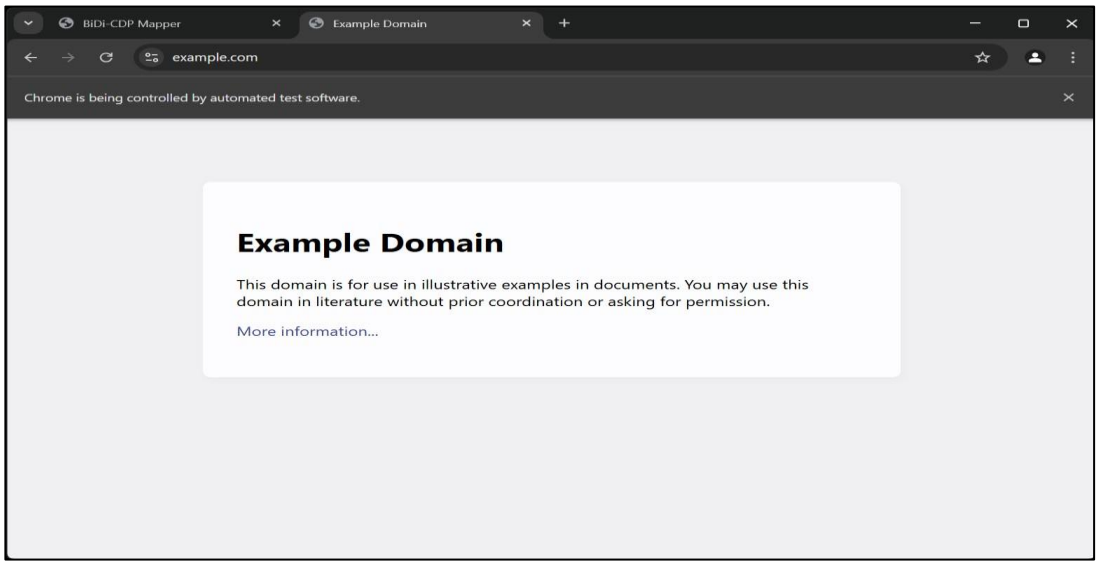
1. Open Katalon Studio and create a **new test case**.
2. Switch to **"Script Mode"** in the test case editor.
3. Write test scripts using Katalon's built-in APIs and Groovy syntax. Example:

```
import com.kms.katalon.core.webui.keyword.WebUiBuiltInKeywords as WebUI

WebUI.openBrowser('https://example.com')

WebUI.setText(findTestObject('Page_Login/txt_Username'), 'testuser')

WebUI.setEncryptedText(findTestObject('Page_Login/txt_Password'), 'encryptedpassword')

WebUI.click(findTestObject('Page_Login/btn_Login'))

WebUI.verifyElementPresent(findTestObject('Page_Home/lbl_Welcome'), 10)

WebUI.closeBrowser()
```

4. Click **"Run"** to execute the script and check test results.




BiDi-CDP Mapper
Example Domains
iana.org/help/example-domains
Chrome is being controlled by automated test software.
iana
Internet Assigned Numbers Authority
Domains Protocols Numbers About
Example Domains
As described in RFC 2606 and RFC 6761, a number of domains such as example.com and example.org are maintained for documentation purposes. These domains may be used as illustrative examples in documents without prior coordination with us. They are not available for registration or transfer.
We provide a web service on the example domain hosts to provide basic information on the purpose of the domain. These web services are provided as best effort, but are not designed to support production applications. While incidental traffic for incorrectly configured applications is expected, please do not design applications that require the example domains to have operating HTTP service.
Further Reading
IANA-managed Reserved Domains
Last revised 2017-05-13.


BiDi-CDP Mapper
Example Domain
example.com
Chrome is being controlled by automated test software.
Example Domain
This domain is for use in illustrative examples in documents. You may use this domain in literature without prior coordination or asking for permission.
More information...

Program 3

Exploring Katalon Recorder Tool: Recorder for Chrome browser, Firefox, Explorer. Play testcases in recorder.

1. Install Katalon Recorder

For Chrome

1. Open **Google Chrome**.
2. Go to the **Chrome Web Store** and search for **Katalon Recorder**.
3. Click **Add to Chrome → Add Extension**.
4. Once installed, you will see the **Katalon Recorder** icon in the browser toolbar.

For Firefox

1. Open **Mozilla Firefox**.
2. Visit the **Firefox Add-ons** store and search for **Katalon Recorder**.
3. Click **Add to Firefox → Add**.
4. After installation, the Katalon Recorder icon will appear in the toolbar.

2. Launching Katalon Recorder

1. Click on the **Katalon Recorder** extension icon in the browser toolbar.
2. The Katalon Recorder interface will open with options to **Record**, **Play**, and **Manage** test cases.

3. Recording a Test Case

1. Click **New** to create a new test case.[in the Katalon Recorder chrome extension]
2. Enter a name for the test case and click **OK**.
3. Click the **Record** button.
4. Navigate to the web application where you want to perform testing.[visit any website of your choice]
5. Perform actions (e.g., clicking buttons, entering text, navigating pages).
6. Stop the recording by clicking the **Stop** button.
7. The recorded steps will appear in the test case editor.

4. Playing Back a Test Case

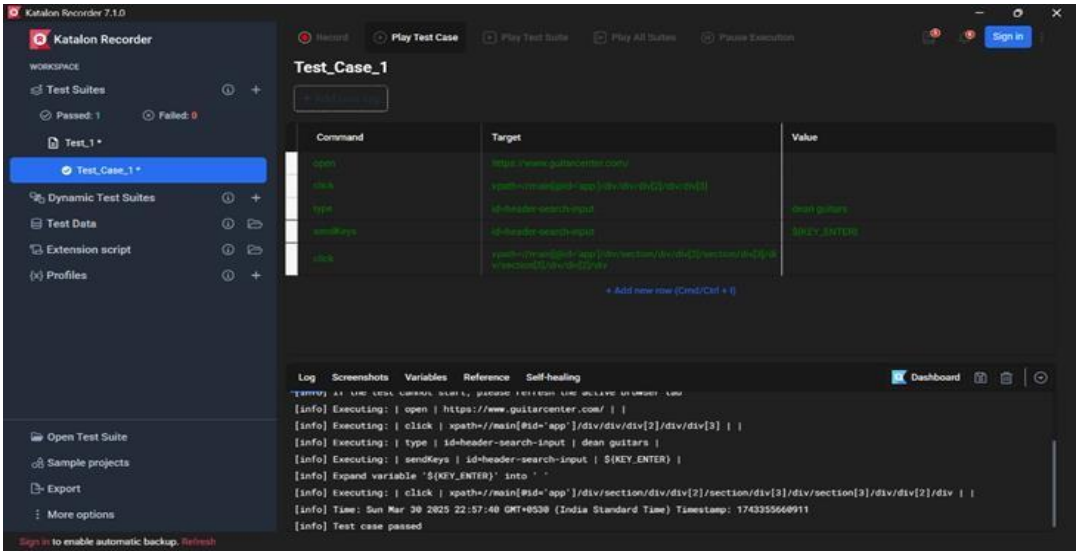
1. Select the recorded test case.
2. Click the **Play** button to execute the test steps.
3. Katalon Recorder will simulate the recorded actions on the browser.
4. Check the **Log** tab for execution results.

5. Exporting and Managing Test Cases

- Click **Export** to save the test case in **Selenium, WebDriver, C#, Python, or Java** formats.
- Use the **Test Suite** feature to group multiple test cases and execute them together.

6. Running Test Cases in Different Browsers

- **Chrome**: Default execution environment.
- **Firefox & Edge**: Requires installing Katalon Recorder for the respective browsers.
- **Explorer (IE)**: Internet Explorer is not supported directly but can be automated using **Katalon Studio**.



Program 4

Exploring Web Spy & Web Object Repository: web spy & object repository. spy on a web object in a browser, Object locators & strategies.

1. **Web Spy** is a tool in Katalon that lets you **inspect and capture web elements** from a browser (like Chrome or Firefox) and **add them to your Object Repository** with a simple click.

What it does:

- Opens a browser and lets you navigate any website.
- Allows you to click on elements to "spy" on them.
- Saves these elements into your test project for reuse.
- Lets you define or modify the **locators** used to identify the element.

---

2. What is the Object Repository**?**

The **Object Repository** is where Katalon stores **Test Objects** — reusable representations of UI elements (buttons, fields, labels, etc.).

Each object includes:

- A **name**.
- One or more **locator strategies** (e.g., XPath, CSS).
- Optional attributes or metadata.

You use these in your test cases with:

groovy
CopyEdit
findTestObject('Object Repository/Page_Login/txt_username')

---

3. How to Spy on a Web Object in a Browser

Steps:

1. Open **Katalon Studio**.
2. Go to Spy Web (from the toolbar or File > New > Spy Web).

3. Select a browser (Chrome is recommended).
4. The browser opens → navigate to your test site (e.g., https://www.saucedemo.com).
5. Hover and click on elements to capture them.
6. On the right panel, you'll see their properties.
7. Click **Add to Object Repository** to save them.

---

4. Object Locators and Strategies

Katalon uses multiple **locators** to identify elements. These can be auto-detected or customized.

Common Strategies:

Strategy	Description	Example
ID	Unique element ID	id=username
Name	HTML name attribute	name=login_form
XPath	XML-like path to an element	//input[@id='username']
CSS	CSS selector	input#username

Katalon Auto-Healing:

If one locator fails (e.g., XPath changes), Katalon can use **backup locators** to recover automatically.

<u>Features</u>

Web Spy: Capture UI elements directly from browser

Object Repository: Store and manage all your UI elements

Locators: Strategies to identify elements in tests

Best Practice: Use IDs, define multiple strategies, always validate.

Program 5

Exploring the Predefined pre-Defined Web UI Keyboards: browser keywords. Windows keywords, checkbox keywords, combo box, textbox. file upload. screenshot, scroll down and scroll up button related, alerts frames, mouse hover. drag and drop and keystroke keywords.

These predefined web UI keywords are commonly used in tools like **Selenium**, **Robot Framework**, **TestComplete**, or **Katalon Studio**.

Browser Keywords

Used to control the browser.

- Open Browser – Opens a new browser instance.
- Close Browser – Closes the current browser.
- Go To – Navigates to a URL.
- Maximize Browser Window – Maximizes the browser.
- Set Window Size – Sets custom size for the window.
- Get Location – Gets the current URL.
- Reload Page – Refreshes the current page.

Window Keywords

Manage browser windows and tabs.

- Switch Window – Switches focus to a specific browser window/tab.
- Close Window – Closes the currently focused window.
- Get Window Titles – Returns a list of all open window titles.

Checkbox Keywords

For interacting with checkboxes.

- Select Checkbox – Checks a checkbox.
- Unselect Checkbox – Unchecks a checkbox.
- Checkbox Should Be Selected – Asserts that checkbox is selected.

- Checkbox Should Not Be Selected – Asserts that checkbox is not selected.

Combo Box / Drop-Down Keywords

Used to select options from a dropdown.

- Select From List By Label – Selects an option using visible text.
- Select From List By Value – Uses the value attribute.
- Select From List By Index – Selects by position.
- Get Selected List Label – Gets the selected label.

Textbox Keywords

Work with input fields.

- Input Text – Inputs text into a text field.
- Clear Element Text – Clears existing text.
- Textfield Value Should Be – Validates the text in the field.

File Upload Keywords

Simulate file selection and upload.

- Choose File – Selects a file from local system to upload.
- Submit Form – Can be used after choosing file, if necessary.

Screenshot Keywords

Capture browser states.

- Capture Page Screenshot – Captures full page screenshot.
- Capture Element Screenshot – Captures a specific element.

Scroll Down and Scroll Up Button Keywords

For vertical navigation.

- Scroll Element Into View – Scrolls to an element.
- Execute JavaScript – Custom JS scroll: window.scrollBy(0,1000) or scrollTo(0, 0)

Alert Frame / Dialog Keywords

Handle popups like alerts or confirmations.

- Handle Alert – Accepts an alert.
- Dismiss Alert – Cancels the alert.
- Alert Should Be Present – Waits/asserts alert presence.
- Input Text Into Alert – Inputs text if alert supports it (prompt).

Mouse Hover Keywords

Simulate mouse movements.

- Mouse Over – Moves mouse over an element (hover).
- Mouse Down / Mouse Up – Used for hold-click operations.

Drag and Drop Keystroke Keywords

Simulate drag-n-drop actions.

- Drag And Drop – Drags an element and drops to target.
- Drag And Drop By Offset – Drags element by pixel offset.

Keystroke/Keyboard Keywords

Simulate keyboard actions.

- Press Keys – Simulates pressing keys (e.g., CTRL+S, ENTER).
- Input Text – Typing into a field.
- Simulate Keystroke – (in some tools) for low-level key events.

Here's a **Katalon Studio-specific** version of the breakdown you asked for. Katalon Studio uses **predefined Web UI keywords** to automate actions on web elements — great for UI testing. Let's explore each of the categories with **Katalon's WebUI class** in mind:

1. Browser Keywords (WebUI)

These control browser sessions:

- WebUI.openBrowser('') – Launches a browser instance.
- WebUI.navigateToUrl('https://example.com') – Opens a URL.
- WebUI.maximizeWindow() – Maximizes browser window.
- WebUI.refresh() – Refreshes the page.
- WebUI.closeBrowser() – Closes the browser.

2. Window Keywords

Control multiple windows or tabs:

- WebUI.switchToWindowTitle('Page Title') – Switch by title.
- WebUI.switchToWindowIndex(1) – Switch using window index.
- WebUI.getWindowTitle() – Gets current window's title.

3. Checkbox Keywords

Interact with checkboxes:

- WebUI.check(findTestObject('Page/Checkbox')) – Check it.
- WebUI.uncheck(findTestObject('Page/Checkbox')) – Uncheck.
- WebUI.verifyElementChecked(findTestObject('Page/Checkbox'), 5) – Assert it's selected.
- WebUI.verifyElementNotChecked(...) – Assert it's not.

4. Combo Box (Dropdown) Keywords

Handle dropdown menus:

- WebUI.selectOptionByLabel(...) – Select using visible label.
- WebUI.selectOptionByValue(...) – Use HTML value.
- WebUI.selectOptionByIndex(...) – Select by position.
- WebUI.getSelectedOptionText(...) – Retrieve selected text.

5. Textbox Keywords

For input fields:

- WebUI.setText(findTestObject('Page/TextField'), 'Sample Input')
- WebUI.clearText(findTestObject('Page/TextField'))
- WebUI.verifyElementAttributeValue(...) – Check input value.

6. File Upload

Simulate file uploads:

- WebUI.uploadFile(findTestObject('Page/UploadButton'), 'C:\\path\\to\\file.png')

7. Screenshot

Take screenshots of the page or element:

- WebUI.takeScreenshot('Screenshots/snap.png')
- WebUI.takeElementScreenshot(findTestObject('Page/Element'))

▼▲ 8. Scroll Up/Down

Scroll to view specific content:

- WebUI.scrollToElement(findTestObject('Page/Footer'), 5) – Scroll into view.
- WebUI.executeJavaScript('window.scrollBy(0,500)', []) – Custom scroll.

⚠ 9. Alert Frame/Pop-up Handling

Handle JS alerts or prompts:

- WebUI.acceptAlert() – Click OK.
- WebUI.dismissAlert() – Click Cancel.
- WebUI.getAlertText() – Read the message.
- WebUI.setAlertText('Hello') – Set value (for prompts).

10. Mouse Hover

Mouse over elements (e.g., to reveal a submenu):

- WebUI.mouseOver(findTestObject('Page/MenuItem'))

11. Drag and Drop

Drag one object to another:

- WebUI.dragAndDropToObject(findTestObject('Source'), findTestObject('Target'))

12. Keystroke Simulation

Simulate typing or keyboard commands:

- WebUI.sendKeys(findTestObject('Page/Input'), Keys.ENTER)
- WebUI.sendKeys(findTestObject('Page/Input'), Keys.chord(Keys.CONTROL, 'a'))

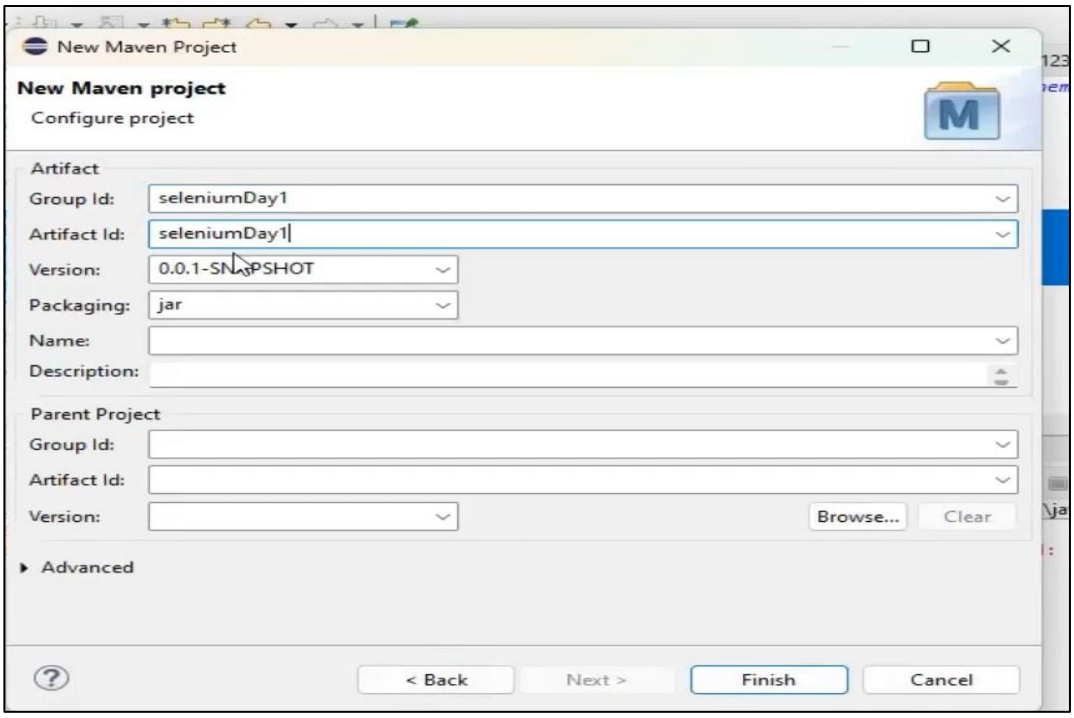
Note: Keys is imported from org.openqa.selenium.Keys.

Program 6

Installation of Selenium: Setting up Selenium RC and Selenium Web Driver.

Step 1 : Create a New Maven Project from File Menu > New.

Step 2 : To create a Maven project, you need to specify **Group ID** and **Artifact ID,** then finish.



Step 3 : Add the following dependency within the **<dependencies>** section of the **pom.xml** file, ensuring that it is placed before the closing **</project>** tag. This dependency is necessary for integrating Selenium with Java in the project.

```
1•<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:
2   <modelVersion>4.0.0</modelVersion>
3   <groupId>Day1</groupId>
4   <artifactId>Day1</artifactId>
5   <version>0.0.1-SNAPSHOT</version>
6
7•<dependencies>
8 <!-- https://mvnrepository.com/artifact/org.seleniumhq.se
9•<dependency>
10     <groupId>org.seleniumhq.selenium</groupId>
11     <artifactId>selenium-java</artifactId>
12     <version>4.28.1</version>
13 </dependency>
14 </dependencies>
15 </project>
```

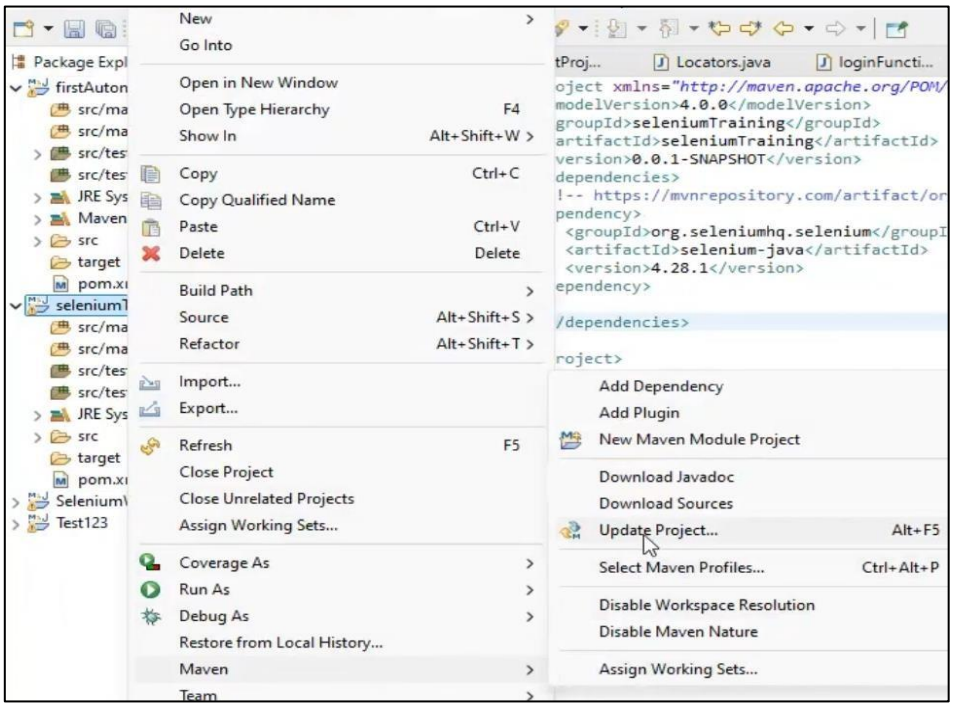
Manual Configuration

•Download Selenium Jars from selenium.dev/downloads/

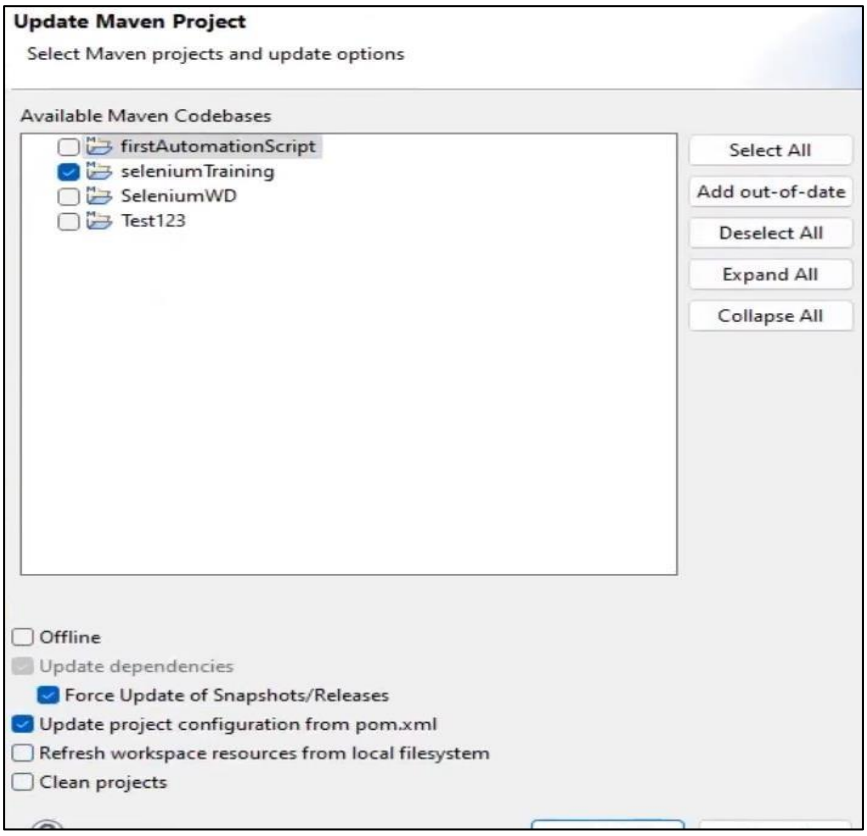
•Extract the file.

•Goto Eclipse -> Right click on the project -> Properties -> Java build path ->Libraries ->Click on Class Path ->Click on Add external jars ->Select jar files -> Apply and Close

Step 4 : Right click on your project > Maven > Update Project



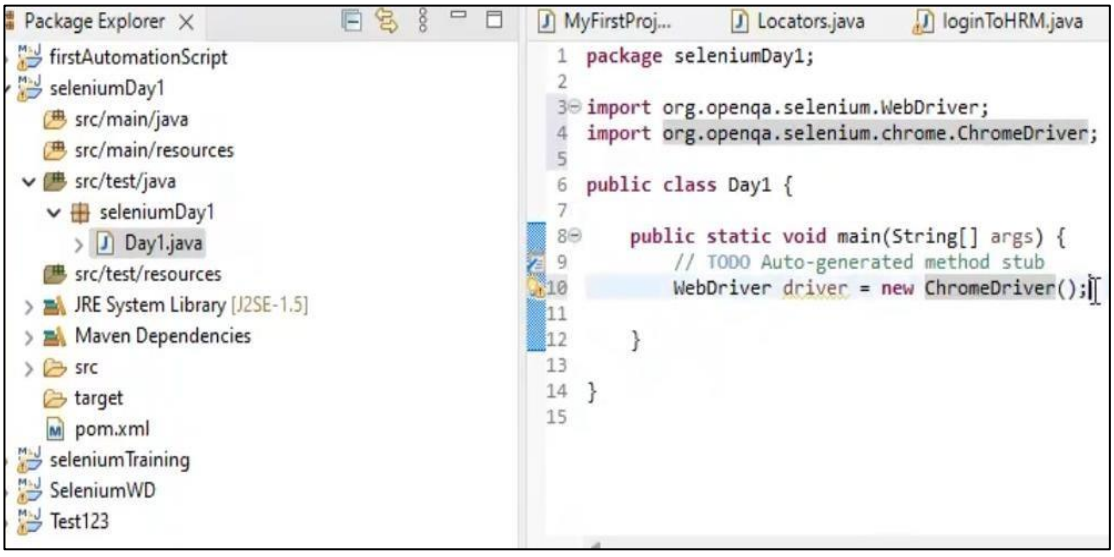
Step 5: Select the following check boxes



Step 6 : Right click on **src/test/java** and create a new package and class with any name.

Select Public static void main while creating class.

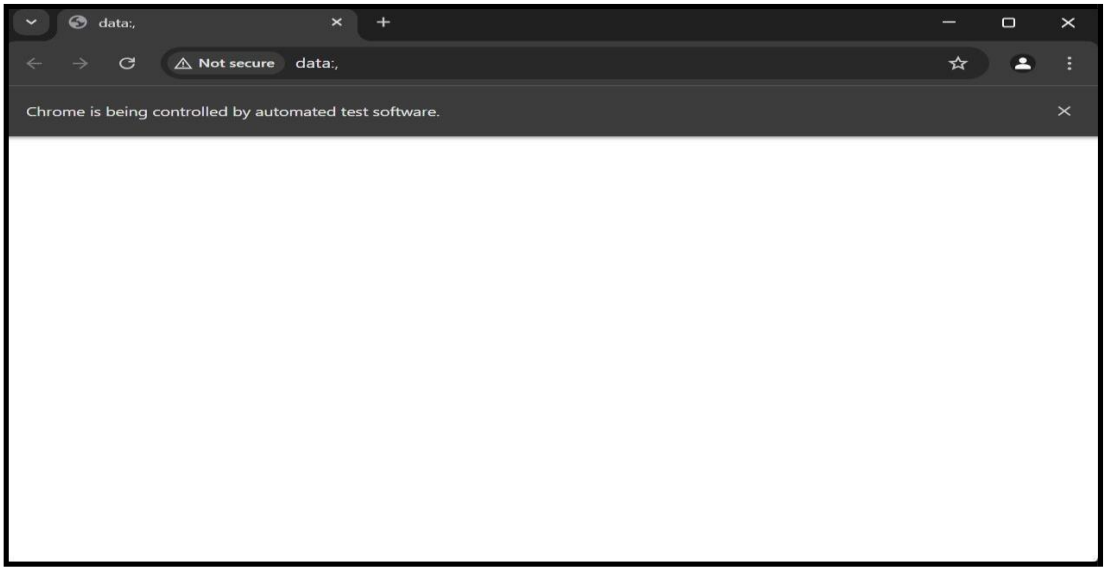
Type the following code in your class.



WebDriver Code:

```
package Prog6;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

public class Day1 {

        public static void main(String args[]) {

                WebDriver driver=new ChromeDriver();

        }

}
```

Output: Chrome is opened through Web driver using Selenium



Program 7

Designing Testcases: Test Case design and Conversion of Mapping and Templates using selenium tool and Manual test cases mapping with Selenium test cases.

Test Case Design:

• Analyze Requirements

• Identify Test Scenarios: Cover positive, negative and edge cases

• Define Test Steps: Write detailed steps, including preconditions, inputs and expected outcomes

• Use Templates for consistency:

Templates consist of : Test case id, title, preconditions, test steps, input, expected result

Mapping: Map each manual test case to its corresponding selenium script.

Test Case Login Template:

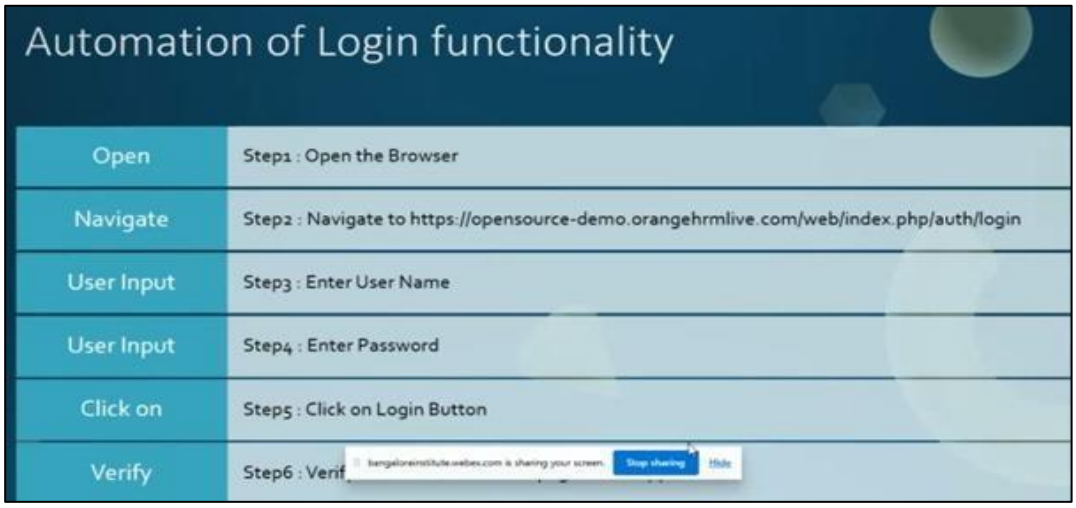
```
@Test
public void testLogin() {
WebDriver driver = new ChromeDriver();
driver.get("https://example.com");
driver.findElement(By.id("username")).sendKeys("testUser");
driver.findElement(By.id("password")).sendKeys("testPass");
driver.findElement(By.id("loginButton")).click();
Assert.assertEquals(driver.getTitle(), "Dashboard");
driver.quit();
}
```

Selenium Login Script:

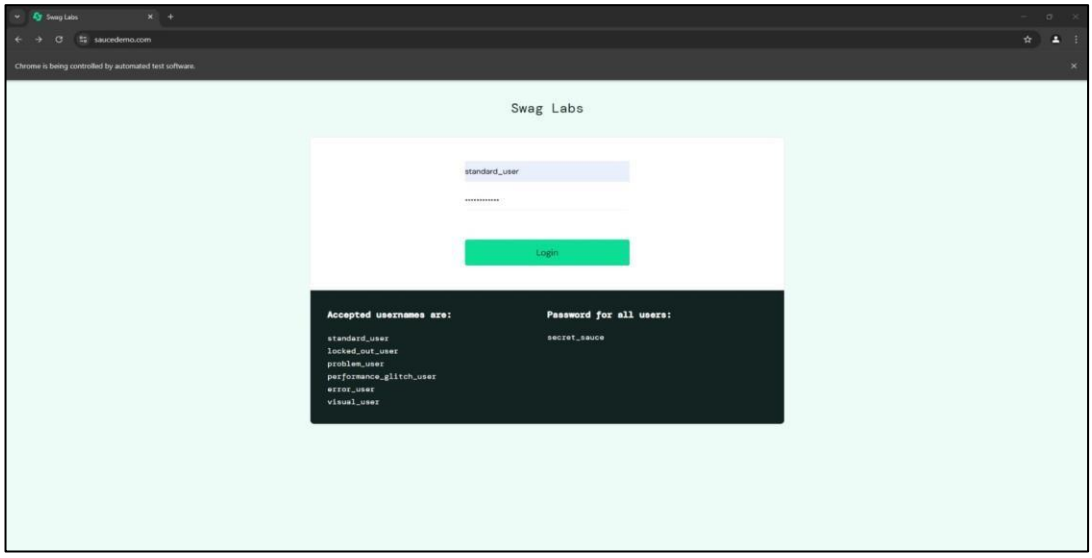
```
import org.openqa.selenium.By;
```

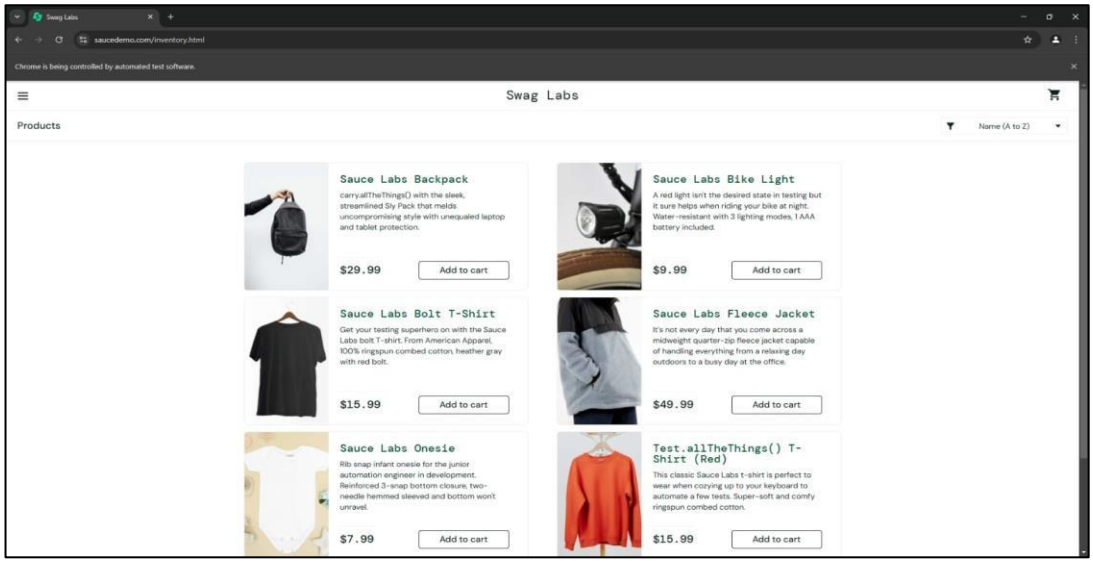
```
import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;


public class aaa {

public static void main(String[] args) {

WebDriver driver = new ChromeDriver();

driver.get("https://www.saucedemo.com/");

driver.findElement(By.id("user-name")).click(); //just for the click function

driver.manage().window().maximize();

driver.findElement(By.id("user-name")).sendKeys("standard_user");

driver.findElement(By.id("password")).sendKeys("secret_sauce1");

driver.findElement(By.className("submit-button")).click();

 boolean flag = driver.findElement(By.xpath("//button[text()=\"Open
Menu\"]")).isDisplayed();

 if (flag == true)

 {System.out.println("Successfully logged in! hurRAY ");}

 /*driver.close(); // terminate test window only

 driver.quit(); //to Terminate all windows and test
```

Automation of Login Functionality:



Output:





Mapping Table: Update the Actual Result based on Output

Expected result	Actual result	Test data	Status
Browser should open			
User should be able to navigate		Username:	
User should be able to enter user name and password		standard_user	
User should be logged in successfully		Password:	
Browser should be closed		secret_sauce	Design

Test case Name	Pre con	Test steps
		Open browser
		Navigate to https://www.saucedemo.com/
		Enter correct user name and password
		Click on Login button
https://www.saucedemo.com/ website is launched and log		Close the browser

Program 8

Using Selenium IDE: Develop a test suite containing minimum 4 test cases.

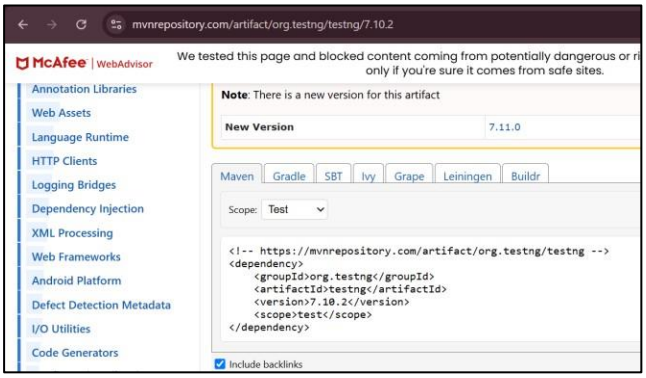
Test Suite: a collection of multiple test cases grouped together for execution. It allows testers to run multiple tests sequentially or in parallel to validate different functionalities of a web application.

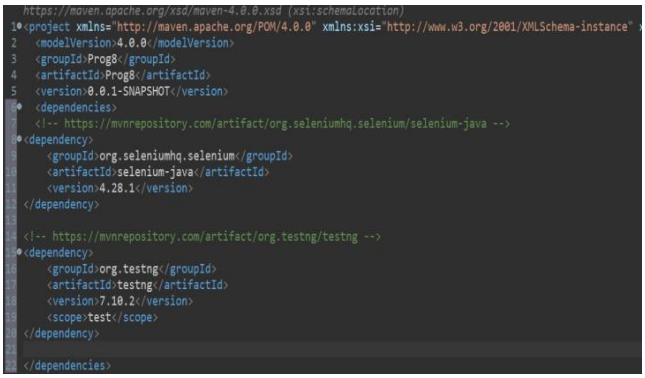
Add Selenium jar files and dependency first.

1. Navigate to https://mvnrepository.com/artifact/org.testng/testng

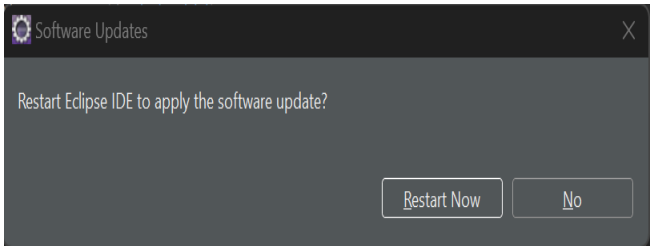
2. Copy the maven dependency

3. In eclipse, copy this under dependencies tag after selenium dependency inside pom.xml

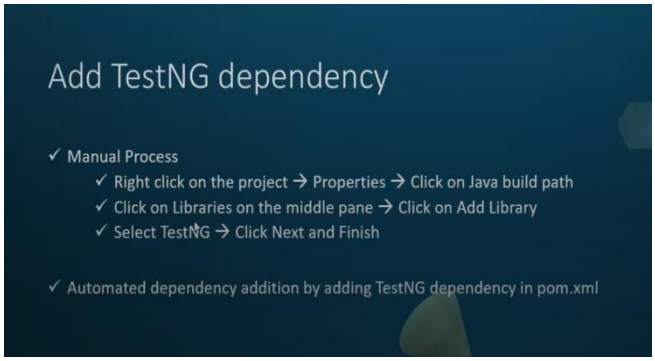


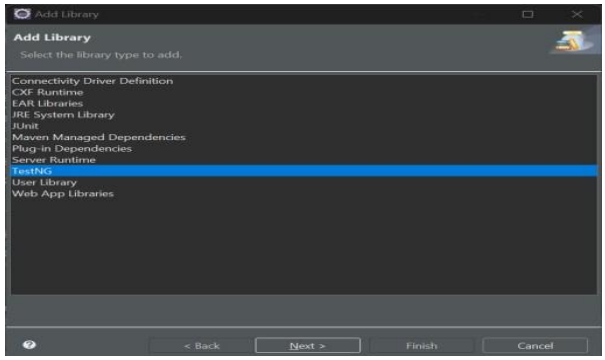


4. Manual Configuration.

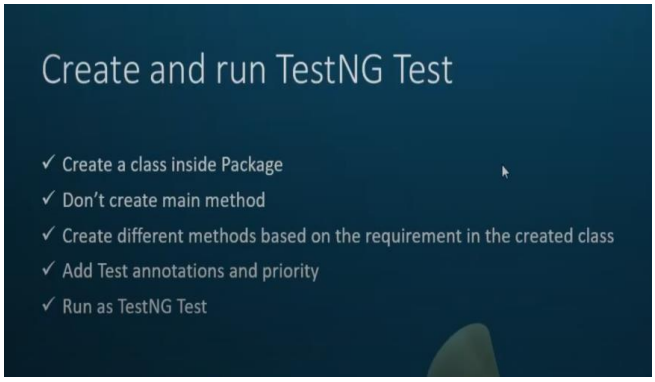


ADDING TESTNG DEPENDENCY:





RUNNING TEST SCRIPT USING TESTNG:



OUTPUT:

```
<terminated> testNGFramework [TestNG] C:\Users\kumku\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_22.0.2.v2
[RemoteTestNG] detected TestNG version 7.11.0
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.
Open browser
Login
Add item to cart
Close browser
PASSED: Prog8.testNGFramework.close
PASSED: Prog8.testNGFramework.login
PASSED: Prog8.testNGFramework.openBrowser
FAILED: Prog8.testNGFramework.addItem
java.lang.AssertionError: Item count mismatch expected [10] but found [5]
```

```
===============================================
    Default test
    Tests run: 4, Failures: 1, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 4, Passes: 3, Failures: 1, Skips: 0
===============================================
```

TEST SUITE WITH 4 TEST CASES:

```
package Prog8;
import org.testng.annotations.Test;
import org.testng.Assert;
public class testNGFramework {
	@Test(priority=1)
	void openBrowser(){
		System.out.println("Open browser");
}
	@Test(priority=2)
	void login(){
		System.out.println("Login");
	}
	@Test(priority=3)
	void addItem(){
		System.out.println("Add item to cart");
		Assert.assertEquals(5,10,"Item count mismatch");
	}
	@Test(priority=4)
	void close(){
		System.out.println("Close browser");
	}
}
```

Program 9

Test Suite: Conduct a test suite for any two web sites.

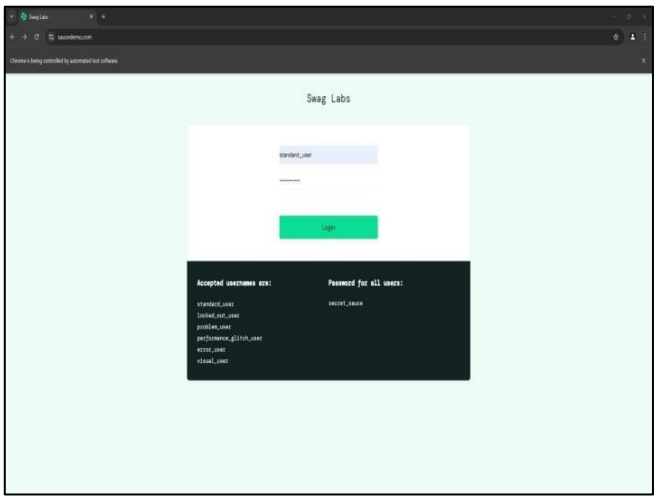
- Add Selenium Jar Files and Dependency
- Add TestNG library and Dependency
- Update Project

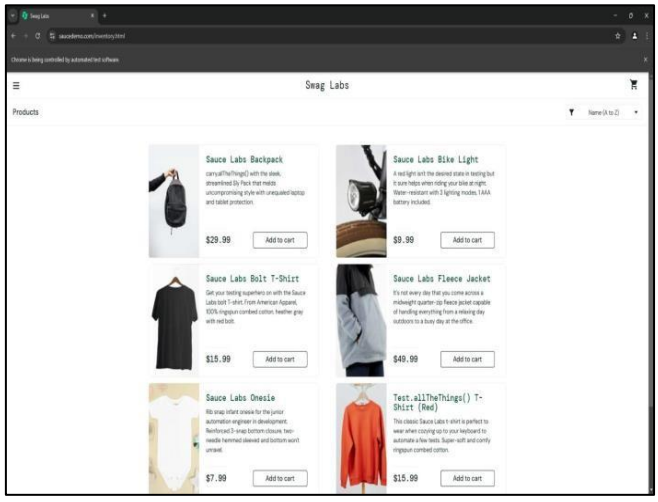
WEBSITE - 1

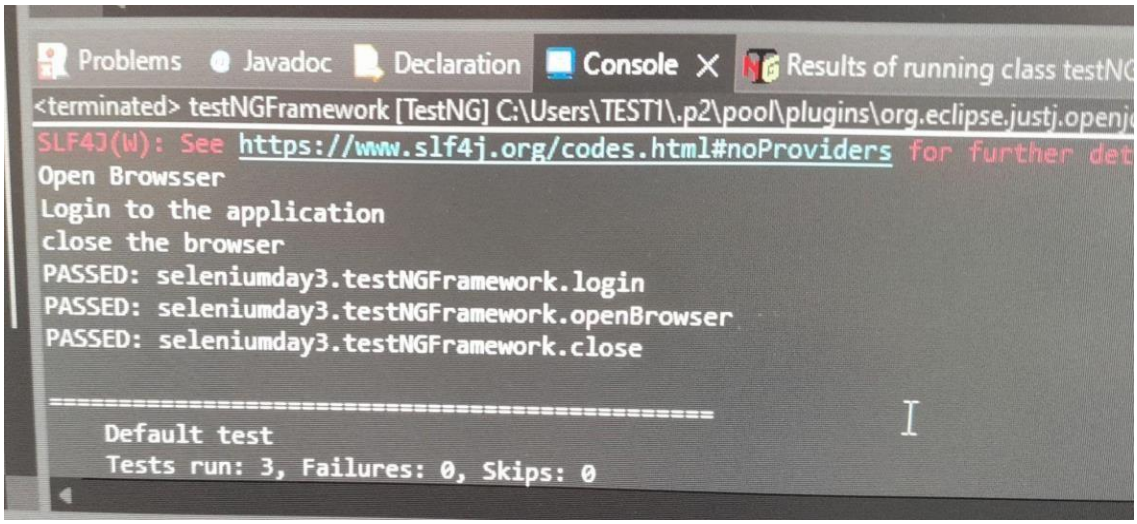
```
package seleniumDay3;
import org.testng.annotations.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class loginWithTestNG {
	WebDriver driver;
	@Test(priority = 1)
	void openBrowser() {
		driver = new ChromeDriver();  // Assigning to class-level driver
		driver.get("https://www.saucedemo.com");
		driver.manage().window().maximize();
}
	@Test(priority = 2)
	void login() {
		driver.findElement(By.id("user-name")).click();
		driver.findElement(By.id("user-name")).sendKeys("standard_user");
		driver.findElement(By.name("password")).sendKeys("secret_sauce");
		driver.findElement(By.id("login-button")).click(); // Corrected name attribute
// Checking if the menu button is displayed after login
		boolean flag = driver.findElement(By.xpath("//button[text()='Open
Menu']")).isDisplayed();
		if (flag) {
```

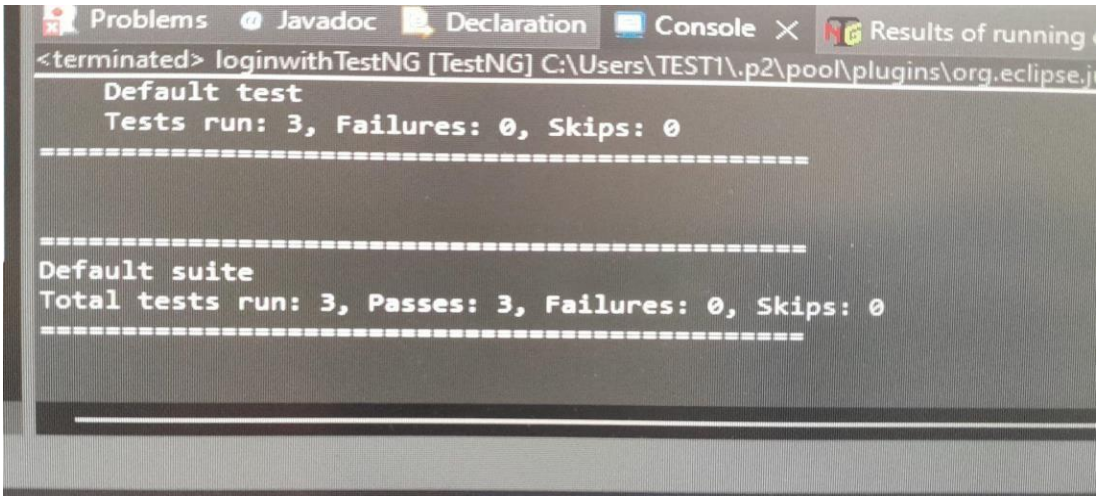
```
System.out.println("User is successfully logged in");
                } else {
                        System.out.println("User is not logged in successfully");
                }
}
        @Test(priority = 3)
        void close() {
                driver.close();
        }
}
```

OUTPUT:







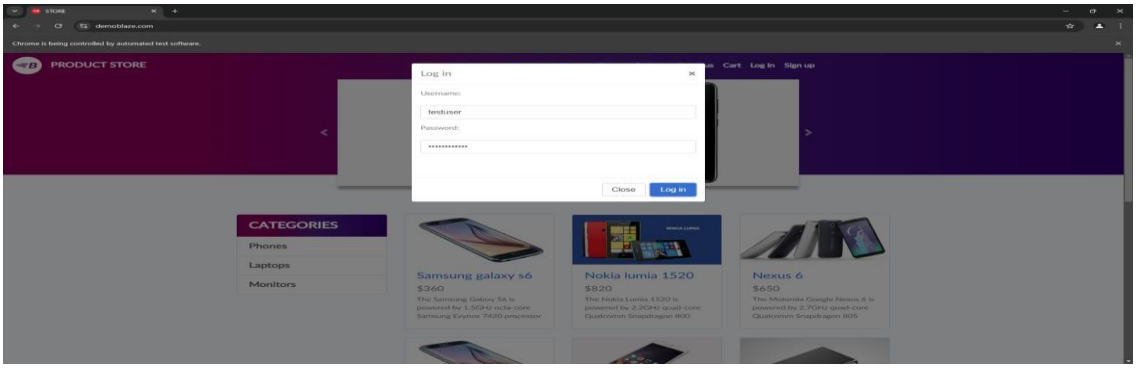


WEBSITE - 2

```
package seleniumDay3;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;

public class LoginDemoBlaze {
    WebDriver driver;
    @BeforeTest
    void openBrowser() {
        driver = new ChromeDriver();
        driver.get("https://www.demoblaze.com/");
        driver.manage().window().maximize();
    }
    @Test(priority = 1)
    void login() throws InterruptedException {
        driver.findElement(By.id("login2")).click();
```

```
Thread.sleep(2000);  // Wait for the login modal to appear
        driver.findElement(By.id("loginusername")).sendKeys("testuser");
        driver.findElement(By.id("loginpassword")).sendKeys("testpassword");
        driver.findElement(By.xpath("//button[text()='Log in']")).click();
        Thread.sleep(3000);  // Wait for login to process
        WebElement logoutButton = driver.findElement(By.id("logout2"));
        if (logoutButton.isDisplayed()) {
            System.out.println("User is successfully logged in");
        } else {
            System.out.println("User login failed");
        }
    }
    @AfterTest
    void closeBrowser() {
        driver.quit();
    }
}
```

OUTPUT:




```
<terminated> LoginDemoBlaze [TestNG] C:\Users\test1\.p2\pool\plugins\org.eclipse.justj.openjdk.ho

===============================================
    Default test
    Tests run: 1, Failures: 1, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 1, Passes: 0, Failures: 1, Skips: 0
===============================================
```

Program 10

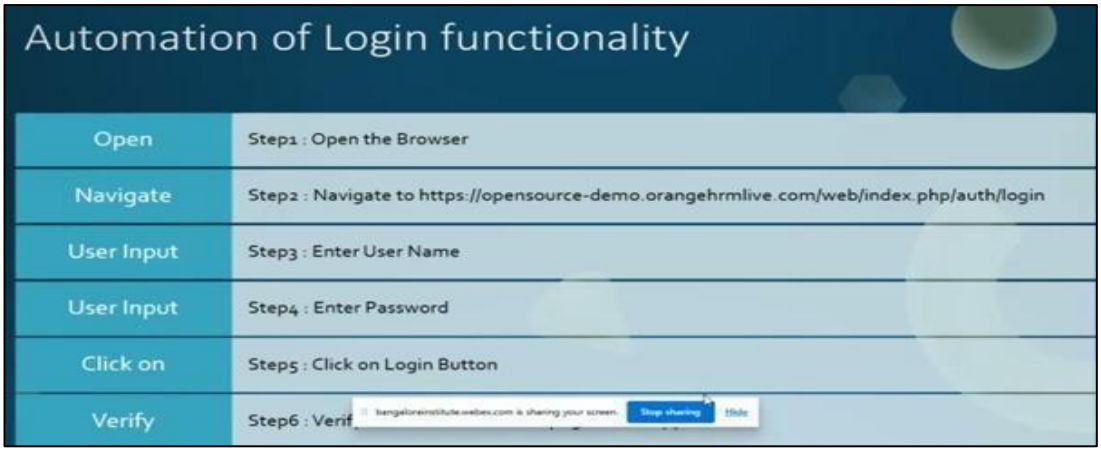
Test Scripts: Develop and test a program to login a specific web page using Selenium test scripts.

- Add Selenium jar and dependency.
- Update Project
- Run the login script

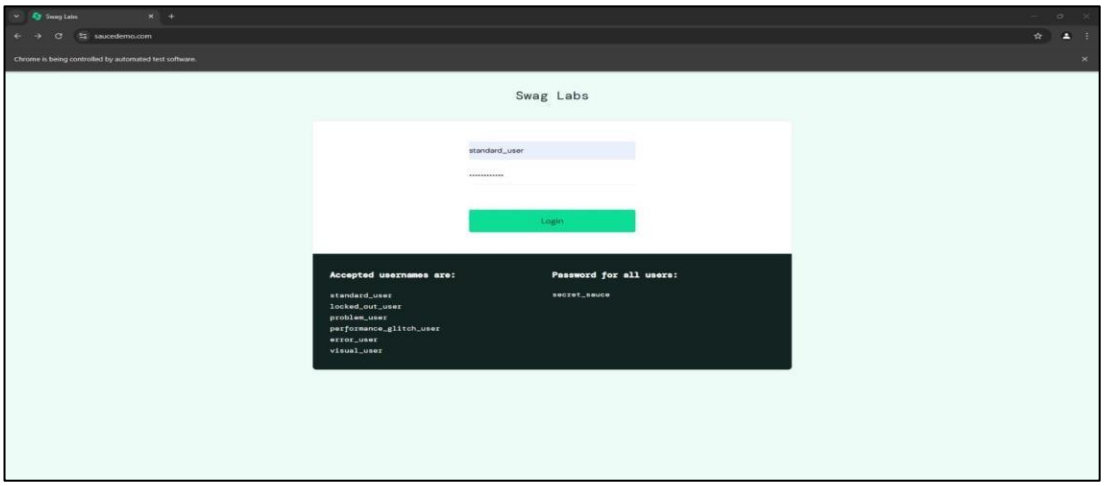
Selenium Login Script:

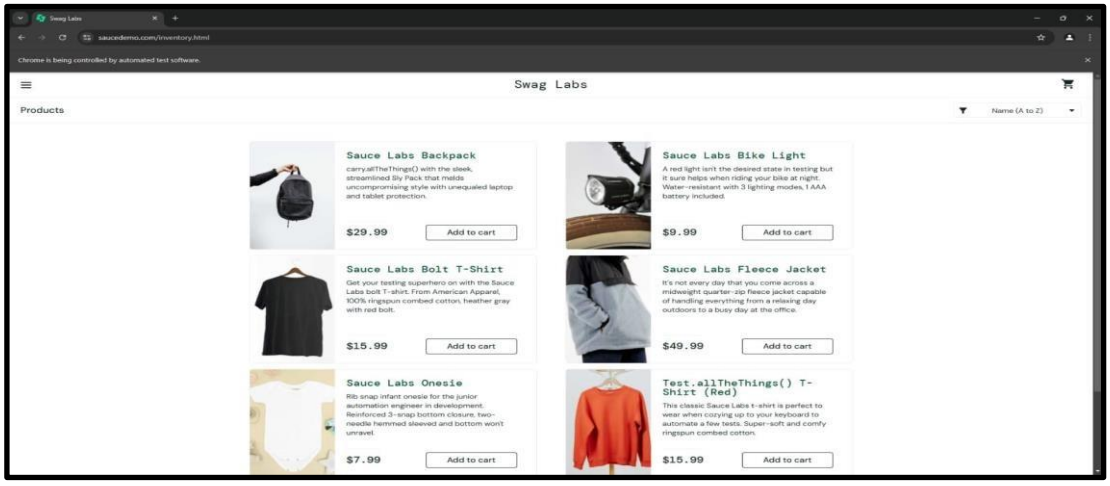
```
import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

public class aaa {

public static void main(String[] args) {

WebDriver driver = new ChromeDriver();

driver.get("https://www.saucedemo.com/");

driver.findElement(By.id("user-name")).click(); //just for the click function

driver.manage().window().maximize();

driver.findElement(By.id("user-name")).sendKeys("standard_user");

driver.findElement(By.id("password")).sendKeys("secret_sauce1");

driver.findElement(By.className("submit-button")).click();

 boolean flag = driver.findElement(By.xpath("//button[text()=\"Open
Menu\"]")).isDisplayed();

 if (flag == true)

  {System.out.println("Successfully logged in! hurRAY ");}

 /*driver.close(); // terminate test window only

 driver.quit(); //to Terminate all windows and test window */

 }
```

Steps:



Output:





Program 11

Test Scripts: Develop and test a program to provide total number of objects present available on the page using selenium test scripts.

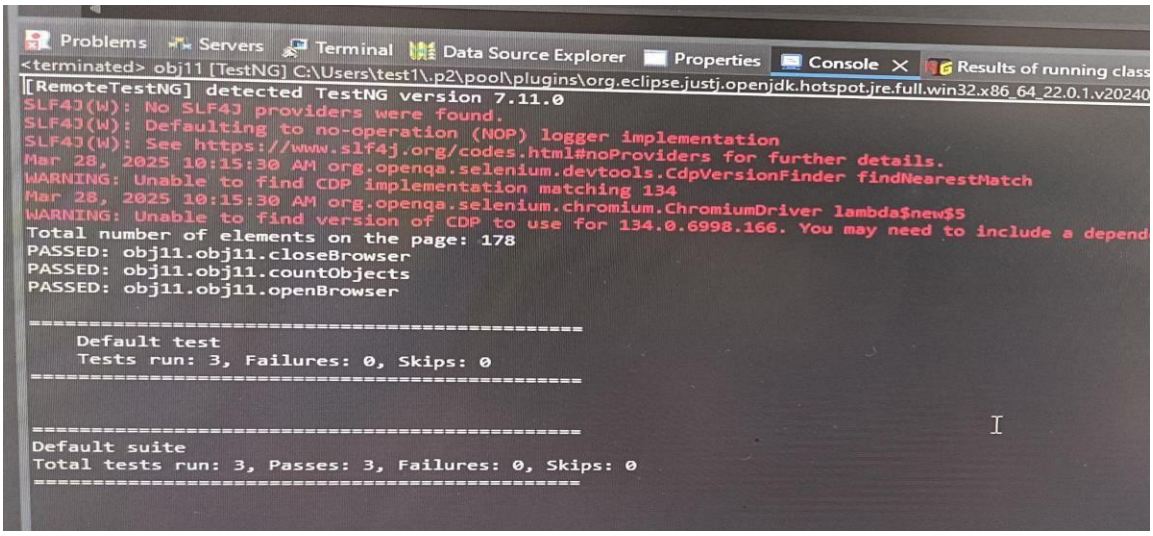
- Add Selenium jar and dependency.
- Add TestNG dependency and library.
- Update Project
- Run the script

Selenium Script to Count No. of Objects:

```
package seleniumDay3;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.Test;
import java.util.List;
public class CountObjects {
   WebDriver driver;

   @Test(priority = 1)
   void openBrowser() {
      driver = new ChromeDriver();
      driver.get("https://www.demoblaze.com/");
      driver.manage().window().maximize();
   }
@Test(priority = 2)
   void countObjects() {
      List<WebElement> elements = driver.findElements(By.xpath("//*"));
      System.out.println("Total number of elements on the page: " + elements.size());
   }
   @Test(Priority = 3)
   void closeBrowser() {
```

```
        driver.quit();
    }
}
```

OUTPUT:



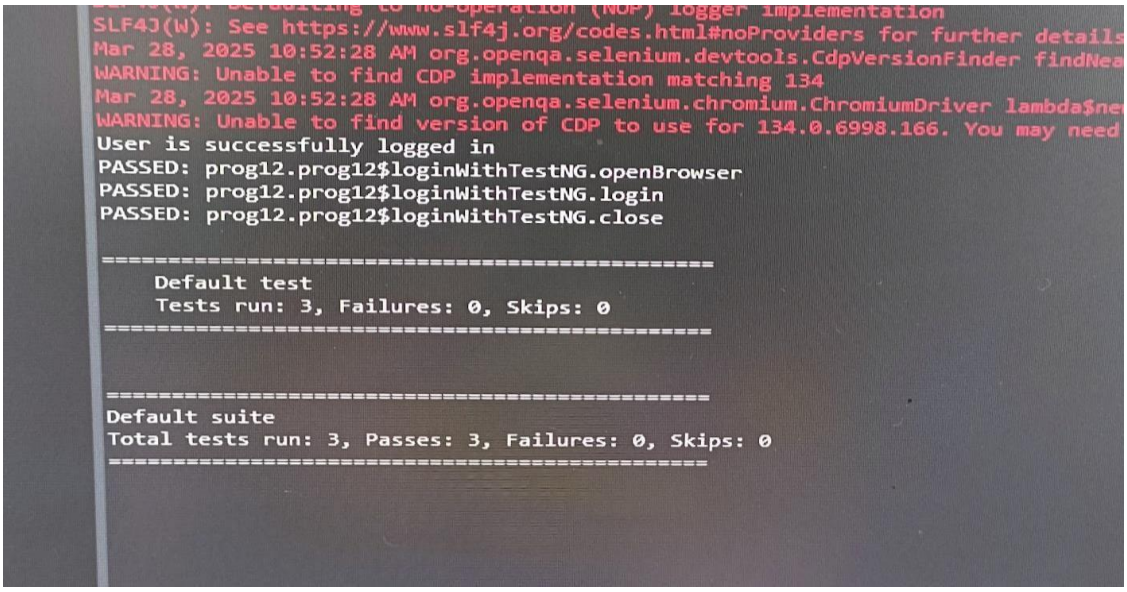
Program 12

Practical Exercise and Wrap-Up: Build Test suit with suitable application and complete end to end automation process, Discussion on Best Practices and Q&A.

- Add Selenium jar and dependency.
- Add TestNG dependency and library.
- Update Project
- Run the script

Selenium Script:

```
package seleniumDay3;
Import org.openqa.selenium.By;
import org.testng.annotations.Test;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class loginWithTestNG {
        WebDriver driver;
        @Test(priority=1)
        void openBrowser(){
                driver = new ChromeDriver();
                driver.get("https://www.saucedemo.com");
                driver.manage().window().maximize();
        }
@Test(priority=2)
        void login() {
                driver.findElement(By.id("user-name")).sendKeys("standard_user");
                driver.findElement(By.name("password")).sendKeys("secret_sauce");

                driver.findElement(By.name("login-button")).click();
         boolean flag= driver.findElement(By.xpath("//button[text()= \"Open
         Menu\"]")).isDisplayed();
                 if(flag){
```

```
System.out.println("User is successfully logged in");
                }
                else{
                        System.out.println("User is not logged in successfully");
                }
        }

        @Test(priority=3)
        void close(){
                driver.close();
        }
    }
```

OUTPUT: